ҚАЗАҚСТАН РЕСПУБЛИКАСЫ ҒЫЛЫМ ЖӘНЕ ЖОҒАРЫ БІЛІМ МИНИСТРЛІГІ

_____ (университет атауы)

_____ (кафедра)

КУРСТЫҚ ЖОБА

«Жобалау паттерндері» (Design Patterns) пәні бойынша

NEXORA — Біртұтас басқару платформасы

Біртұтас операциялар, ақылды жұмыс үдерістері, нақты уақыттағы хабарландырулар

Он GoF паттерні нақты бизнес-логикада: Singleton, Factory Method, Abstract Factory, Observer, State, Strategy, Facade, Mediator, Chain of Responsibility, Command

Орындаған: _____ (топ: _____)

Тексерген: _____

Алматы, 2026

Мазмұны

1. Кіріспе 4
1.1. Жобаның мақсаты 4
1.2. Міндеттер 4
1.3. Өзектілігі 4
2. Жобаның сипаттамасы 6
2.1. Бес бизнес-модуль 6
2.2. Модульдер арасындағы интеграция 6
2.3. Рөлдер 7
3. Технологиялық стек 9
4. Архитектура 10
4.1. Жалпы құрылым 10
4.2. Сервердің модульдері 10
4.3. Модульдер тәуелділігі (циклсіз) 10
4.4. Транзакциялар және бәсекелес сұраныстардан қорғау 11
5. Дерекқор 12
6. Жобалау паттерндері 13
6.1. Singleton (Жалғыз объект) 13
6.2. Factory Method (Фабрикалық әдіс) 15
6.3. Abstract Factory (Абстрактілі фабрика) 18
6.4. Observer (Бақылаушы) 20
6.5. State (Күй) 23
6.6. Strategy (Стратегия) 28
6.7. Facade (Қасбет) 30
6.8. Mediator (Делдал) 32
6.9. Chain of Responsibility (Жауапкершілік тізбегі) 35
6.10. Command (Команда) 39
6.11. Ұқсас паттерндерді салыстыру 41
7. Қауіпсіздік 42
7.1. Авторизация қатесін табу және түзету 42
8. Тестілеу 43
9. Іске қосу және тоқтату 46
9.1. Старт.bat 46
9.2. Стоп.bat 46
10. Қорғауға арналған демо-сценарий 47
10.1. Негізгі сценарий: Қойма → Observer → Құжат айналымы 47

10.2. Әуе билетін брондау..... 47
10.3. Емханаға жазылу 48
11. Оқытушының ықтимал сұрақтары және жауаптар 49
12. Қорытынды..... 51
Белгілі шектеулер 51

1. Кіріспе

Бұл есепте «Жобалау паттерндері» пәні бойынша әзірленген **NEXORA — біртұтас басқару платформасы** сипатталады. Жоба — толыққанды full-stack веб-қосымша: Java/Spring Boot негізіндегі сервер бөлігі, React/TypeScript негізіндегі клиент бөлігі, MySQL дерекқоры, Docker арқылы іске қосу, автоматты тесттер және құжаттама.

Жобаның басты ерекшелігі — он GoF (Gang of Four) паттерні жеке «демонстрациялық» кластарда емес, **нақты бизнес-логиканың ішінде** қолданылады. Әр паттерн шынайы мәселені шешеді, сайттағы әрекеттер арқылы көрінеді, автоматты тесттермен тексеріледі және «Паттерндер зертханасы» бетінде сервердегі нақты кластарды іске қосу арқылы көрсетіледі.

1.1. Жобаның мақсаты

Мақсаты — бір-бірімен оқиғалар арқылы байланысқан бес бизнес-модульден тұратын біртұтас платформа жасау және сол платформада он жобалау паттернінің нақты пайдасын көрсету.

1.2. Міндеттер

1. Қойма, емхана, әуе билеттерін брондау, құжат айналымы және хабарландыру модульдерін бір жүйеге біріктіру.
2. Әр модульде сәйкес паттерндерді шынайы сценарийлерде қолдану (бір паттерн — бір нақты мәселе).
3. Рөлдік қол жеткізу, транзакциялар, қос брондаудан қорғау, аудит журналы сияқты сенімділік талаптарын орындау.
4. Заманауи, ыңғайлы, толық қазақ тіліндегі интерфейс жасау (390, 768, 1280 px ендеріне бейімделген).
5. Паттерндерді және бизнес-сценарийлерді автоматты тесттермен тексеру.
6. Жобаны бір батырмамен іске қосу (Старт.bat) және қауіпсіз тоқтату (Стоп.bat) мүмкіндігін беру.

1.3. Өзектілігі

Нақты кәсіпорындарда склад, медициналық қызмет, логистика және құжат айналымы жеке жүйелерде жүргізіледі, ал олардың арасындағы байланыс қолмен жасалады. NEXORA бір модульдегі оқиға (мысалы, тауар қорының азаюы) басқа модульдерде автоматты түрде жұмыс үдерісін (ескерту, хабарландыру, толықтыру өтінімі, бекіту) іске қосатын архитектураны көрсетеді. Мұндай байланыстыруды дұрыс жобалау үшін дәл осы паттерндер қажет: Observer модульдерді бір-біріне

тәуелсіз етеді, Facade және Mediator күрделі сценарийлерді басқарады, State және Chain of Responsibility бизнес-ережелерді қатаң сақтайды.

2. Жобаның сипаттамасы

2.1. Бес бизнес-модуль

1-кесте — Платформаның бизнес-модульдері

Модуль	Негізгі мүмкіндіктері	Қолданылатын паттерндер
Қойма	Тауарлар, санаттар, жеткізушілер, SKU, сатып алу және бөлшек баға, қалдық, ең аз қор, кіріс/шығыс/түзету қозғалыстары, қозғалыс журналы, төмен қор ескертулері, қор құны, сүзгілер, сұрыптау, беттеу	Command, Observer, Strategy, Singleton
Емхана	Пациенттер, дәрігерлер, мамандықтар, апталық кесте, бос уақыт аралықтары, қабылдауға жазылу, ауыстыру, болдырмау, күйлер (Жоспарланған, Расталған, Аяқталған, Болдырылмаған, Келмеді), қабылдау тарихы	Facade, Observer, Command
Әуе рейстері	Рейстерді іздеу (қала, күн, жолаушы саны), үш сұрыптау түрі, орын картасы, орындарды 15 минутқа ұстау, растау (төлемсіз), болдырмау, мерзімінің өтуі, брондау коды, растау құжаты	State, Strategy, Facade, Mediator, Command
Құжат айналымы	5 құжат түрі, қаралама, бекітуге жіберу, кезекті бекіту маршруты, бекіту/қабылдамау (себебімен)/пысықтауға қайтару, тарих, нұсқалар, пікірлер, визуалды маршрут	Chain of Responsibility, State, Mediator, Command, Facade, Observer
Хабарландырулар	Хабарландыру орталығы, оқылмағандар, маңыздылық деңгейлері (Ақпарат, Сәтті, Ескерту, Сыни), арна баптаулары, жеткізу журналы, қайталау, нақты уақыттағы жаңарту (SSE)	Observer, Strategy, Factory Method, Abstract Factory

2.2. Модульдер арасындағы интеграция

Модульдер бір-бірін тікелей шақырмайды — олар **оқиғалар** арқылы байланысады. Төмендегі сценарийлер нақты жұмыс істейді:

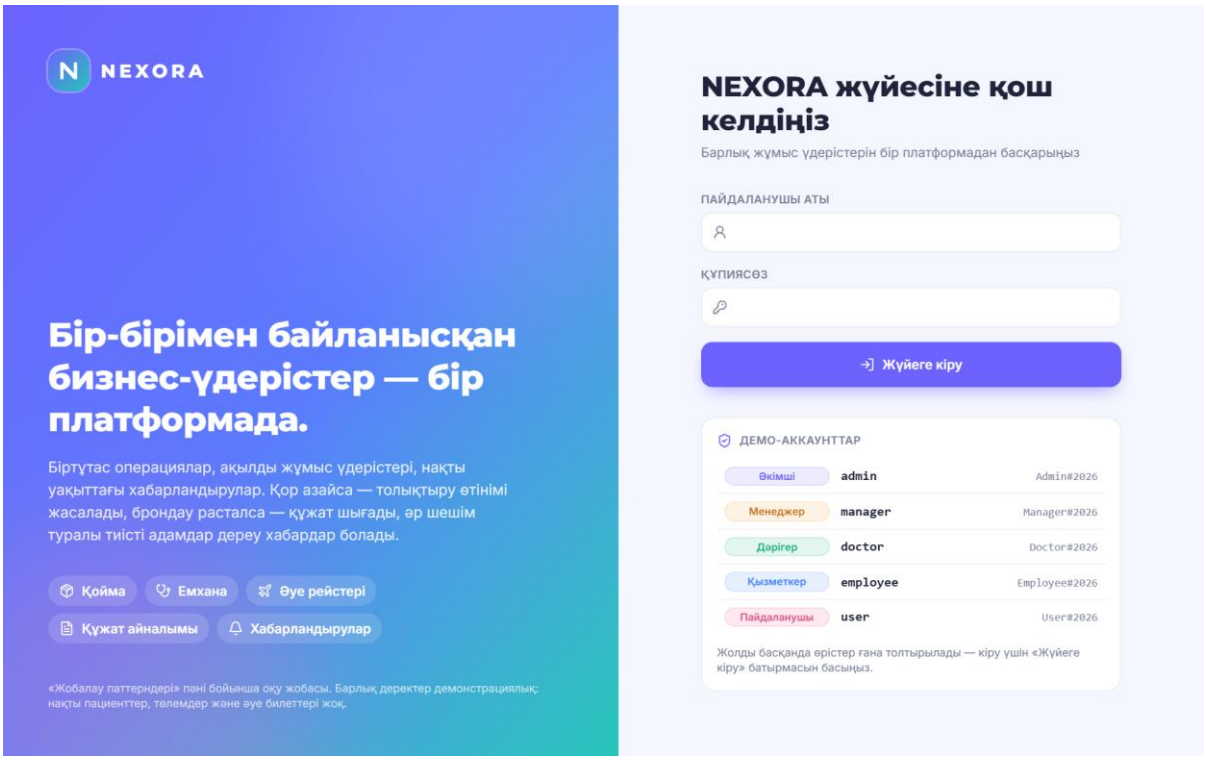
- **Қор азайды** → `LowStockEvent` → ескерту жазылады, хабарландыру жіберіледі, журналға жазба түседі, басқару панелі жаңарады; қызметкер бір батырмамен толықтыру өтінімін жасайды, ол бекіту маршрутынан өтеді.
- **Пациент жазылды** → `AppointmentBookedEvent` → пациент пен дәрігерге хабарландыру, басқару панеліндегі көрсеткіштер жаңарады.
- **Брондау расталды** → құжат айналымында «Брондауды растау құжаты» автоматты жасалады (сол Chain of Responsibility пен State арқылы) → `FlightBookingConfirmedEvent` → хабарландыру.
- **Құжат бекітілді немесе қабылданбады** → `DocumentApprovedEvent` / `DocumentRejectedEvent` → автор мен келесі бекітушіге хабарландыру.

2.3. Рөлдер

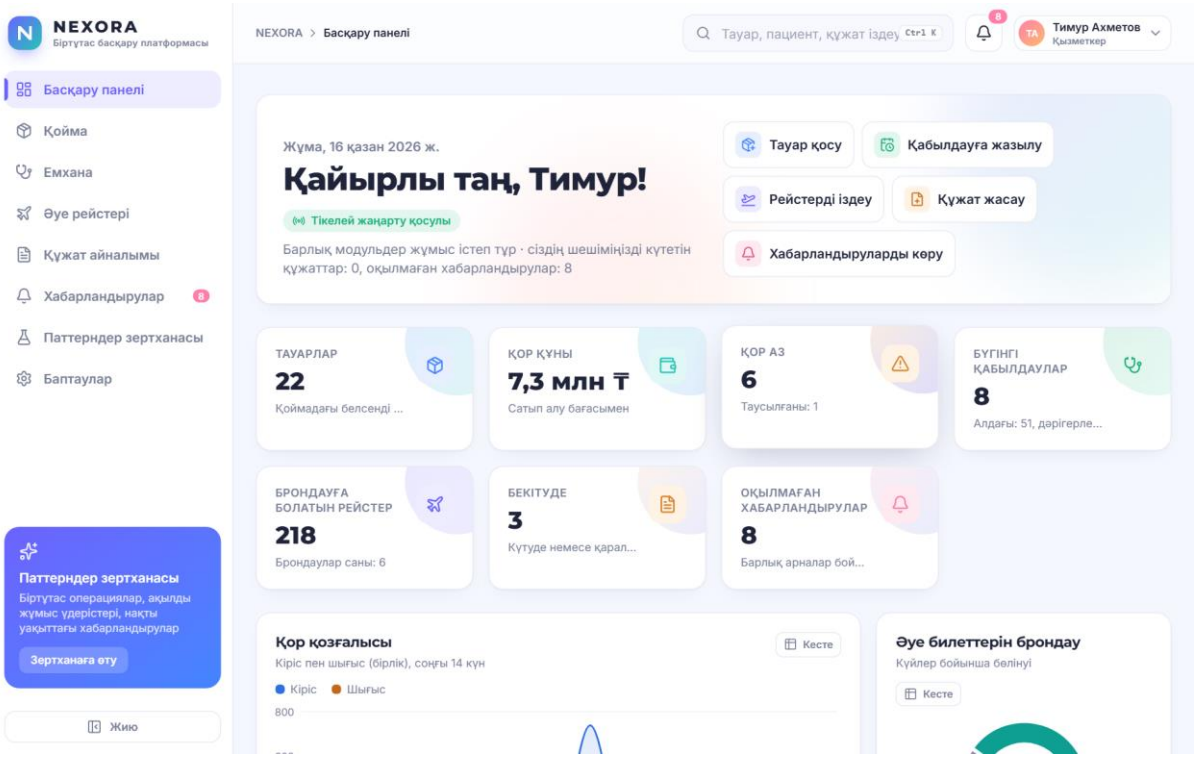
2-кесте — Пайдаланушы рөлдері және өкілеттіктері

Рөл	Өкілеттіктері
Әкімші (ADMIN)	Барлық модульдер, пайдаланушыларды басқару, әрекеттер журналы, кез келген бекіту қадамы (соңғы бекіту)
Менеджер (MANAGER)	Құжаттарды бекіту, қойма, барлық брондаулар, әрекеттер журналы
Дәрігер (DOCTOR)	Өз кестесі мен қабылдаулары, клиникалық күйлерді қою, медициналық есеп жасау
Қызметкер (EMPLOYEE)	Қойма операциялары, тіркеу бөлімі (пациенттер, жазылулар), құжаттар жасау
Пайдаланушы (USER)	Тек өз брондаулары, өз жазылулары және хабарландырулары

Рөлдер **серверде** тексеріледі (`@PreAuthorize` және сервистердегі нысан деңгейіндегі тексерулер). Интерфейстегі батырмаларды жасыру тек ыңғайлылық үшін, қорғаныс емес.



1-сурет — Жүйеге кіру беті (демо-аккаунттар блогы)



2-сурет — Басқару панелі: KPI карточкалары және графиктер

3. Технологиялық стек

3-кесте — Қолданылған технологиялар

Қабат	Технологиялар	Не үшін
Сервер	Java 25, Spring Boot 3.5.16 (Web, Data JPA, Validation, Security, Actuator, Mail), Hibernate 6	REST API, бизнес-логика, транзакциялар, қауіпсіздік
Дерекқор	MySQL 8.4, Flyway (V1–V4 миграциялары), H2 (тек тесттерде)	Реляциялық сұлба, миграциялар, демо деректер
API құжаттамасы	springdoc-openapi 2.8 (Swagger UI)	API-ды интерактивті тексеру
Клиент	React 19, TypeScript 6, Vite 8, Tailwind CSS 4, React Router 8	SPA интерфейсі
Деректер және формалар	TanStack Query 5, React Hook Form, Zod 4	API кэші, формаларды тексеру
Визуализация	Recharts 3, Lucide иконкалары	Графиктер, иконкалар
Тесттер	JUnit 5, Mockito, Spring Boot Test, MockMvc, Vitest, Testing Library	Автоматты тексеру
Инфрақұрылым	Docker, Docker Compose, Nginx, Старт.bat / Стоп.bat	Бір батырмамен іске қосу

4. Архитектура

4.1. Жалпы құрылым

Жүйе үш контейнерден тұрады: **mysql** (дерекқор), **backend** (Spring Boot) және **frontend** (Nginx статикалық SPA файлдарын береді және `/api` сұраныстарын backend-ке жібереді). Браузер үшін фронтенд пен API бір мекенжайда (http://localhost:8088) орналасқан, сондықтан cookie-сеанс пен CSRF-қорғау CORS-сыз жұмыс істейді. Нақты уақыттағы хабарландырулар Server-Sent Events (SSE) арқылы жіберіледі.

4.2. Сервердің модульдері

4-кесте — Backend модульдері (com.nexora пакеті)

Модуль	Жауапкершілігі
shared	Ортақ баптаулар (Singleton PlatformSettings), қателер, оқиғалар (Observer), Command инфрақұрылымы, қауіпсіздік, қазақша мәтіндер (Kk)
auth	Пайдаланушылар, кіру/шығу, рөлдерді басқару, демо-аккаунттарды қалпына келтіру
audit	Әрекеттер журналы, оқиғаларды журналға жазатын бақылаушы
inventory	Тауарлар, қозғалыстар, ескертулер, қор стратегиялары
clinic	Пациенттер, дәрігерлер, кесте, жазылулар, PatientAppointmentFacade
flight	Рейстер, орындар, брондау, State, Strategy, Mediator, BookingFacade
workflow	Құжаттар, Chain of Responsibility, State, Mediator, командалар, DocumentWorkflowFacade
notification	Хабарландырулар, Abstract Factory, Factory Method, арна стратегиясы, SSE
dashboard	Басқару панелінің агрегаттары
lab	«Паттерндер зертханасы» сценарийлері (нақты сервистерді шақырады)

Әр модуль ішінде қабаттар бөлінген: `controller` → `facade`/`service` → `repository` → `entity`; API-ға тек `dto` объектілері шығады (JPA сущностьтары сыртқа берілмейді). Паттерндерге арналған пакеттер: `command`, `state`, `strategy`, `mediator`, `chain`, `channel`, `delivery`.

4.3. Модульдер тәуелділігі (циклсіз)

- Бизнес-модульдер (inventory, clinic, flight, workflow) хабарландыру, аудит немесе басқару панеліне **тәуелді емес** — олар тек `shared.event` пакетіндегі оқиғаларды жариялайды.
- Flight модуліне растау құжаты керек, бірақ ол workflow-ты білмейді: `ConfirmationDocumentPort` интерфейсін (порт) жариялайды, ал workflow оны

жүзеге асырады (`BookingConfirmationDocumentAdapter`). Бұл — тәуелділікті инверсиялау.

- `CommandInvoker` (shared) аудитке `AuditTrail` порты арқылы жазады, оны audit модулі жүзеге асырады.

4.4. Транзакциялар және бәсекелес сұраныстардан қорғау

5-кесте — Конкуренттік қауіптер және шешімдер

Қауіп	Шешім
Бір тауарды бір уақытта екі рет есептен шығару	`SELECT … FOR UPDATE` (PESSIMISTIC_WRITE) + `@Version`
Бір дәрігерге бір уақытқа екі жазылу	Дәрігер жолын құлыптау, содан кейін [start, end) аралықтарының қабаттасуын дәрігер мен пациент үшін тексеру; 4 параллель сұраныспен тексерілген — тек біреуі сәтті
Соңғы орынды екі адам брондау	Рейс жолын құлыптау + `UNIQUE(flight_id, seat_number)` + `@Version`
Бір құжатты бір уақытта бекіту	Құжат жолын құлыптау + `@Version`
Команда ортасындағы қате	`CommandInvoker` транзакцияны толық кері қайтарады, сәтсіздік аудитке бөлек транзакцияда жазылады

1-листинг — Дәрігер жолын құлыптау (қос жазылудан қорғау) (backend/src/main/java/com/nexora/clinic/repository/DoctorRepository.java)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select d from Doctor d where d.id = :id")
Optional<Doctor> lockById(@Param("id") Long id);
```

2-листинг — Уақыт аралықтарының қабаттасуын тексеру сұранысы (backend/src/main/java/com/nexora/clinic/repository/AppointmentRepository.java)

```
/** Half-open interval overlap: [start, end) intersects [s, e) iff start < e and end > s. */
@Query("""
        select count(a) from Appointment a
        where a.doctor.id = :doctorId and a.status in :statuses
          and a.startTime < :end and a.endTime > :start
          and (:excludeId is null or a.id <> :excludeId)
        """)
long countDoctorOverlaps(@Param("doctorId") Long doctorId, @Param("start") Instant start, @Param("end")
Instant end,
                         @Param("statuses") Collection<AppointmentStatus> statuses, @Param("excludeId")
Long excludeId);
```

5. Дерекқор

Сұлба толығымен **Flyway** миграцияларымен басқарылады; Hibernate `validate` режимінде жұмыс істейді, яғни сущностьтар мен кестелер сәйкес келмесе, қосымша іске қосылмайды. Уақыт UTC түрінде `DATETIME(6)` бағандарында сақталады, ақша `DECIMAL(12,2)` ↔ `BigDecimal`, құпиясөздер тек BCrypt хеші түрінде.

6-кесте — Flyway миграциялары

Нұсқа	Мазмұны
V1__schema.sql	Барлық кестелер, сыртқы кілттер, индекстер, бірегейлік және CHECK шектеулері
V2__demo_data.java	Модульдер арасында келісілген демо деректер (орнату күніне қатысты күндер)
V3__kazakh_localization.java	Демо деректерді қазақ тіліне аудару (тек UPDATE, деректер жойылмайды)
V4__kazakh_localization_fixes.java	Аударманың қосымша түзетулері

7-кесте — Негізгі кестелер

Кесте	Мақсаты
users, audit_logs	Пайдаланушылар (рөлі, BCrypt хеші), әрекеттер журналы
categories, suppliers, products, inventory_transactions, stock_alerts	Қойма: анықтамалықтар, тауарлар, өзгермейтін қозғалыс журналы, ескертулер
specialties, doctors, doctor_schedules, patients, appointments	Емхана: дәрігерлер, апталық кесте, пациенттер, қабылдаулар
airports, flight_routes, flights, flight_seats, flight_bookings, booking_passengers, booking_documents	Әуе рейстері: бағыттар, рейстер, орындар, брондаулар, растау құжаттары
documents, approval_steps, document_actions, document_comments, document_revisions	Құжат айналымы: құжаттар, бекіту қадамдары, тарих, пікірлер, нұсқалар
notifications, notification_preferences, delivery_attempts	Хабарландырулар, арна баптаулары, жеткізу әрекеттері

6. Жобалау паттерндері

Бұл бөлім — есептің негізгі бөлігі. Әр паттерн үшін: анықтамасы, шешетін мәселесі, жобадағы нақты кластары, **жобадан алынған нақты код**, жұмыс сценарийі, неге дәл осы паттерн таңдалғаны, ұқсас паттерндерден айырмашылығы, тексеретін тест және қорғауда қалай көрсету керегі берілген. Код листингтері жобаның бастапқы файлдарынан автоматты түрде алынған.

8-кесте — Он паттерннің жиынтық кестесі

№	Паттерн	Түрі	Негізгі класс	Қай жерде
1	Singleton	Туындатушы	PlatformSettings	Барлық модульдер
2	Factory Method	Туындатушы	DeliveryHandlerCreator	Хабарландырулар
3	Abstract Factory	Туындатушы	NotificationChannelFactory	Хабарландырулар
4	Observer	Мінез-құлықтық	DomainEvent + Listener-лер	Барлық модульдер арасында
5	State	Мінез-құлықтық	BookingState, DocumentState	Әуе рейстері, Құжаттар
6	Strategy	Мінез-құлықтық	FlightSortStrategy т.б.	Рейстер, Қойма, Хабарландырулар
7	Facade	Құрылымдық	BookingFacade, PatientAppointmentFacade, DocumentWorkflowFacade	Рейстер, Емхана, Құжаттар
8	Mediator	Мінез-құлықтық	FlightBookingMediator, DocumentWorkflowMediator	Рейстер, Құжаттар
9	Chain of Responsibility	Мінез-құлықтық	DocumentCheckHandler	Құжаттар
10	Command	Мінез-құлықтық	Command, CommandInvoker	Барлық пайдаланушы әрекеттері

> **IDE-де паттернді қалай тез табу керек**
> IntelliJ IDEA-да `Ctrl+N` (класс бойынша іздеу) арқылы класс атын теріңіз; интерфейсте `Ctrl+Alt+B` — барлық жүзеге асыруларын көрсетеді; `Alt+F7` — қай жерде қолданылатынын көрсетеді. Әр негізгі кластың Javadoc түсіндірмесінде «Design pattern: …» жазуы бар, оны `Ctrl+Shift+F` арқылы іздеуге болады.

6.1. Singleton (Жалғыз объект)

Анықтамасы

Singleton — класстың тек **бір ғана** экземпляры болуын қамтамасыз етіп, оған жаһандық қол жеткізу нүктесін беретін туындатушы паттерн.

Шешетін мәселесі

Бизнес-үдерістердің ережелері (брондауды ұстау уақыты, бекіту шектері, төмен қор саясаты, уақыт белдеуі, e-mail қосулы ма) бірнеше модульде қажет. Әр модуль өз көшірмесін жасаса, ережелер бір-бірінен алшақтап кетуі мүмкін еді. Сондықтан ережелер **бір ортақ, өзгермейтін** объектіде сақталады.

Жобадағы кластар

- `shared/config/PlatformSettings` — Singleton (Spring-тің әдепкі singleton scope бины).
- `shared/config/NexoraProperties` — `application.yml` ішіндегі `nexora.*` баптауларын байланыстыру.
- Қолданушылар: `InventoryStrategyResolver`, `ManagerApprovalHandler`, `AdditionalApprovalHandler`, `BookingRecorder`, `ScheduleService`, `EmailChannelFactory`, `DashboardController`.

3-листинг — PlatformSettings — бір ортақ, өзгермейтін баптаулар объектісі (backend/src/main/java/com/nexora/shared/config/PlatformSettings.java)

```
@Component
public class PlatformSettings {


    private final String platformName;
    private final ZoneId businessZone;
    private final Duration bookingHold;
    private final int appointmentSlotMinutes;
    private final BigDecimal managerApprovalThreshold;
    private final BigDecimal additionalApprovalThreshold;
    private final String lowStockPolicy;
    private final int flightScheduleHorizonDays;
    private final boolean emailEnabled;
    private final String emailFrom;
    private final String instanceId = UUID.randomUUID().toString();
    private final Instant createdAt = Instant.now();


    public PlatformSettings(NexoraProperties properties) {
        this.platformName = properties.platformName();
        this.businessZone = ZoneId.of(properties.businessTimezone());
        this.bookingHold = Duration.ofMinutes(properties.bookingHoldMinutes());
        this.appointmentSlotMinutes = properties.appointmentSlotMinutes();
        this.managerApprovalThreshold = properties.managerApprovalThreshold();
        this.additionalApprovalThreshold = properties.additionalApprovalThreshold();
        this.lowStockPolicy = properties.lowStockPolicy();
        this.flightScheduleHorizonDays = properties.flightScheduleHorizonDays();
        this.emailEnabled = properties.emailEnabled();
        this.emailFrom = properties.emailFrom();
    }
```

4-листинг — Singleton-ды қолдану: белсенді қор стратегиясын таңдау (backend/src/main/java/com/nexora/inventory/strategy/InventoryStrategyResolver.java)

```
public LowStockDetectionStrategy activeDetection() {
    return detection.get(settings.lowStockPolicy());
}
```

5-листинг — Экземплярдың жалғыз екенін көрсету (Паттерндер зертханасы) (backend/src/main/java/com/nexora/lab/LabService.java)

```
public Map<String, Object> singleton() {
    PlatformSettings a = context.getBean(PlatformSettings.class);
    PlatformSettings b = context.getBean(PlatformSettings.class);
```

```
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("injectedInstanceId", settings.instanceId());
    m.put("lookupAInstanceId", a.instanceId());
    m.put("lookupBInstanceId", b.instanceId());
    m.put("identityHashCodes", List.of(System.identityHashCode(settings), System.identityHashCode(a),
System.identityHashCode(b)));
    m.put("sameInstance", a == b && b == settings);
```

Қалай жұмыс істейді

`PlatformSettings` класы `@Component` ретінде белгіленген, сондықтан Spring оны **бір рет** жасайды және барлық тәуелді кластарға конструктор арқылы **сол бір экземплярды** береді. Барлық өрістері `final`, сеттерлері жоқ — объект өзгермейді, сондықтан көп ағынды ортада қауіпсіз. Ешкім `new PlatformSettings(...)` деп өзі жасамайды. Зертханада `ApplicationContext.getBean()` екі рет шақырылып, үшеуінің `instanceId` және `identityHashCode` мәндері бірдей екені көрсетіледі.

Неге осы паттерн

Баптаулар — жүйе бойы ортақ және өзгермейтін мәндер; оларды әр жерде жеке жасау мағынасыз әрі қауіпті. Spring singleton — классикалық Singleton-ның қазіргі, тестілеуге ыңғайлы нұсқасы: тестте `new PlatformSettings(...)` арқылы кез келген мәнмен жасауға болады, ал тәуелділіктер жасырылмайды.

> **Қорғауда міндетті түрде айту керек**
> Spring singleton — бұл **бір ApplicationContext ішінде бір экземпляр**, бүкіл JVM-ге бір экземпляр емес. Классикалық GoF Singleton (жеке конструктор + статикалық өріс) бір ClassLoader-ге бір экземпляр береді. Сондай-ақ Singleton-да пайдаланушының немесе нақты транзакцияның өзгермелі күйі сақталмайды — ол үшін әр сұранысқа жаңа объектілер (`BookingContext`, `WorkflowContext`) жасалады.

Тексеру

`PlatformIntegrationTest.platformSettingsIsASingleSharedInstance` — контекстен екі рет алынған бин бір объект екенін және зертхана API-і `sameInstance: true` қайтаратынын тексереді.

6.2. Factory Method (Фабрикалық әдіс)

Анықтамасы

Factory Method — объект жасау интерфейсін базалық класта анықтап, **қандай нақты объект жасалатынын ішкі кластарға** шешуге беретін туындатушы паттерн.

Шешетін мәселесі

Хабарландыруды кез келген арна арқылы жеткізу алгоритмі бірдей: өңдеушіні жасау → іске қосу → жеткізу әрекетін (`DeliveryAttempt`) реттік нөмірімен сақтау.

Тек өңдеушінің өзі (және оны қайталауға бола ма) әртүрлі. Алгоритмді бір рет жазып, өңдеушіні жасауды ішкі кластарға қалдыру керек.

Жобадағы кластар

9-кесте — Factory Method қатысушылары

Рөлі	Класс
Creator (абстрактты жасаушы)	`DeliveryHandlerCreator` — `deliver()` (тұрақты алгоритм), `createHandler()` (фабрикалық әдіс)
Concrete Creator	`InAppDeliveryHandlerCreator`, `EmailDeliveryHandlerCreator`
Product (өнім)	`DeliveryHandler` интерфейсі
Concrete Product	`InAppDeliveryHandler` (қайталанбайды), `EmailDeliveryHandler` (қайталауға болады)

6-листинг — Creator: тұрақты алгоритм және фабрикалық әдіс (backend/src/main/java/com/nexora/notification/delivery/DeliveryHandlerCreator.java)

```
public abstract class DeliveryHandlerCreator {

    private final DeliveryAttemptRepository attempts;
    private final Clock clock;

    protected DeliveryHandlerCreator(DeliveryAttemptRepository attempts, Clock clock) {
        this.attempts = attempts;
        this.clock = clock;
    }

    public abstract NotificationChannel channel();

    /** The factory method. */
    protected abstract DeliveryHandler createHandler();

    public final DeliveryAttempt deliver(Notification notification, User recipient) {
        DeliveryHandler handler = createHandler();
        SendResult result;
        try {
            result = handler.handle(notification, recipient);
        } catch (RuntimeException ex) {
            result = SendResult.failed("Жеткізудегі күтпеген қате: " + ex.getMessage());
        }
        int attemptNo = attempts.lastAttemptNo(notification.getId(), channel()) + 1;
        return attempts.save(new DeliveryAttempt(notification.getId(), channel(), result.status(),
attemptNo,
                result.detail(), clock.instant()));
    }

    public final boolean retryable() {
        return createHandler().retryable();
    }
}
```

7-листинг — Concrete Creator: e-mail өңдеушісін жасау (backend/src/main/java/com/nexora/notification/delivery/EmailDeliveryHandlerCreator.java)

```
@Component
public class EmailDeliveryHandlerCreator extends DeliveryHandlerCreator {

    private final EmailChannelFactory family;
```

```
    public EmailDeliveryHandlerCreator(DeliveryAttemptRepository attempts, Clock clock,
EmailChannelFactory family) {
        super(attempts, clock);
        this.family = family;
    }

    @Override public NotificationChannel channel() { return NotificationChannel.EMAIL; }

    @Override protected DeliveryHandler createHandler() { return new EmailDeliveryHandler(family); }
}
```

8-листинг — Product интерфейсі (backend/src/main/java/com/nexora/notification/delivery/DeliveryHandler.java)

```
public interface DeliveryHandler {

    SendResult handle(Notification notification, User recipient);

    /** Whether a failed or skipped attempt may be retried safely. */
    boolean retryable();
}
```

Қалай жұмыс істейді

`NotificationDispatcher` әр арна үшін сәйкес creator-ды картадан алып, `creator.deliver(notification, user)` шақырады. `deliver()` ішінде `createHandler()` шақырылады — `InAppDeliveryHandlerCreator` `InAppDeliveryHandler` береді, `EmailDeliveryHandlerCreator` — `EmailDeliveryHandler`. Нәтиже `delivery_attempts` кестесіне жазылады. «Қайталау» батырмасы тек e-mail үшін көрінеді, өйткені тек `EmailDeliveryHandler.retryable()` `true` қайтарады.

Неге осы паттерн және айырмашылығы

Жаңа арна (мысалы, SMS) қосу үшін бір жаңа creator және бір жаңа handler жазу жеткілікті — `deliver()` алгоритмі мен диспетчер өзгермейді. **Abstract Factory-ден айырмашылығы:** Factory Method мұрагерлік арқылы **бір өнім** жасайды (ішкі класс әдісті қайта анықтайды); Abstract Factory композиция арқылы өзара үйлесімді **өнімдер отбасын** жасайды. Жобада екеуі бірге жұмыс істейді: Factory Method жасаған өңдеуші ішінде Abstract Factory отбасын қолданады.

Тексеру

`NotificationFactoriesTest`: `emailIsSkippedHonestlyWhenSmtpIsNotConfigured`, `emailIsDeliveredOnlyWhenSmtpAcceptsIt`, `inAppCreatorProducesNonRetryableHandler`.

6.3. Abstract Factory (Абстрактілі фабрика)

Анықтамасы

Abstract Factory — нақты кластарды атамай-ақ, **бір-бірімен үйлесетін объектілер отбасын** жасауға арналған интерфейс беретін туындатушы паттерн.

Шешетін мәселесі

Әр хабарландыру арнасына үш үйлесімді компонент керек: **форматтаушы** (тостқа арналған қысқа мәтін немесе сәлемдесуі бар хат), **жіберуші** (SSE арқылы push немесе SMTP) және **қолжетімділік тексерушісі**. E-mail форматтаушысын in-app жіберушісімен араластыруға болмайды. Клиент код нақты кластарды білмеуі керек.

10-кесте — Abstract Factory отбасылары

Абстрактілі өнім	InApp отбасы	Email отбасы
MessageFormatter	InAppMessageFormatter (қысқа мәтін)	EmailMessageFormatter (тақырып, сәлемдесу, қолтаңба)
MessageSender	InAppMessageSender (кіріс жәшігі + SSE)	EmailMessageSender (SMTP)
ChannelAvailabilityChecker	InAppAvailabilityChecker	EmailAvailabilityChecker (SMTP бапталған ба)

9-листинг — Абстрактілі фабрика интерфейсі (backend/src/main/java/com/nexora/notification/channel/NotificationChannelFactory.java)

```
public interface NotificationChannelFactory {

    NotificationChannel channel();

    MessageFormatter createFormatter();

    MessageSender createSender();

    ChannelAvailabilityChecker createAvailabilityChecker();
}
```

10-листинг — Нақты фабрика: e-mail отбасы (backend/src/main/java/com/nexora/notification/channel/email/EmailChannelFactory.java)

```
@Component
public class EmailChannelFactory implements NotificationChannelFactory {

    private final ObjectProvider<JavaMailSender> mailSender;
    private final PlatformSettings settings;

    public EmailChannelFactory(ObjectProvider<JavaMailSender> mailSender, PlatformSettings settings) {
        this.mailSender = mailSender;
        this.settings = settings;
    }

    @Override public NotificationChannel channel() { return NotificationChannel.EMAIL; }
    @Override public MessageFormatter createFormatter() { return new
EmailMessageFormatter(settings.platformName()); }
```

```
    @Override public MessageSender createSender() { return new EmailMessageSender(mailSender,
settings); }
    @Override public ChannelAvailabilityChecker createAvailabilityChecker() {
        return new EmailAvailabilityChecker(mailSender, settings);
    }
}
```

11-листинг — Клиент: тек абстрактілі интерфейстермен жұмыс істейді (backend/src/main/java/com/nexora/notification/delivery/AbstractDeliveryHandler.java)

```
public abstract class AbstractDeliveryHandler implements DeliveryHandler {


    private final NotificationChannelFactory family;


    protected AbstractDeliveryHandler(NotificationChannelFactory family) {
        this.family = family;
    }


    @Override
    public SendResult handle(Notification notification, User recipient) {
        Availability availability = family.createAvailabilityChecker().check(recipient);
        if (!availability.available()) {
            return SendResult.skipped(availability.reason());
        }
        var message = family.createFormatter().format(notification, recipient);
        return family.createSender().send(message, notification, recipient);
    }
}
```

12-листинг — Шынайы жіберуші: SMTP қабылдамаса, «жіберілді» демейді (backend/src/main/java/com/nexora/notification/channel/email/EmailMessageSender.java)

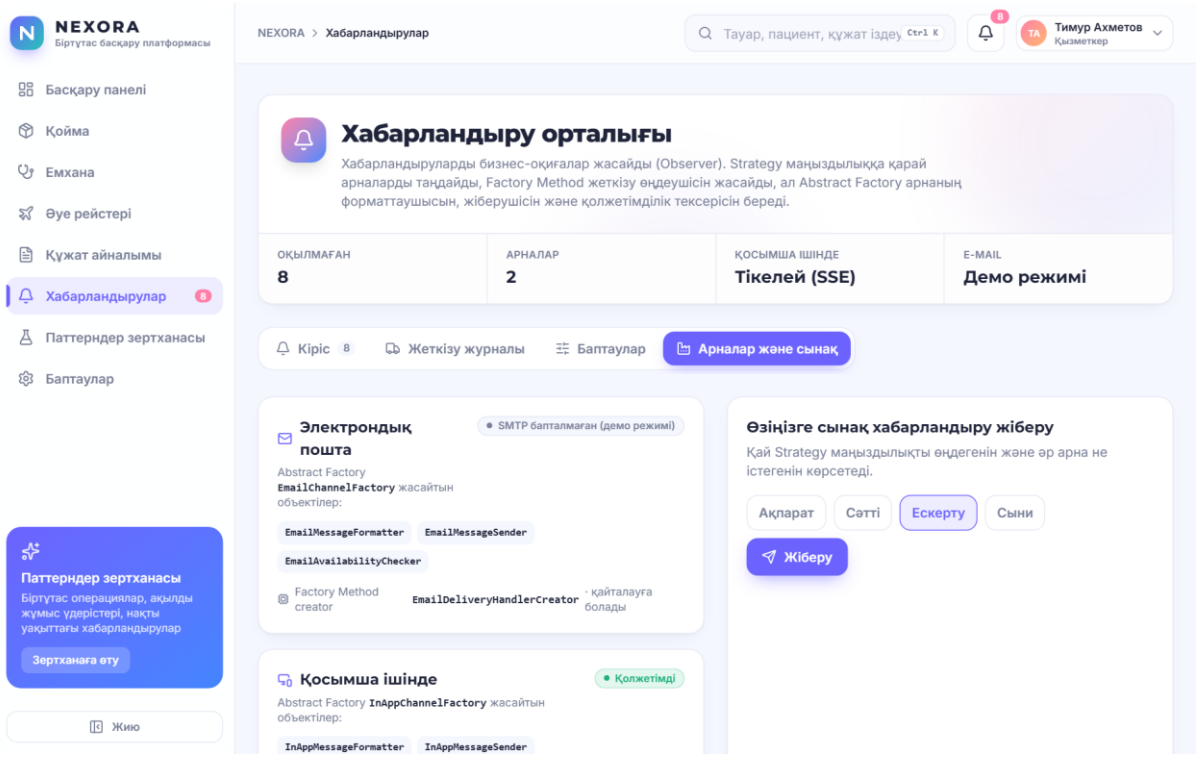
```
public SendResult send(FormattedMessage message, Notification n, User recipient) {
    JavaMailSender sender = mailSender.getIfAvailable();
    if (sender == null || !settings.emailEnabled()) {
        return SendResult.skipped("SMTP бапталмаған (демо режимі) - хат жіберілмеді");
    }
    try {
        SimpleMailMessage mail = new SimpleMailMessage();
        mail.setFrom(settings.emailFrom());
        mail.setTo(recipient.getEmail());
        mail.setSubject(message.subject());
        mail.setText(message.body());
        sender.send(mail);
        return SendResult.delivered("SMTP сервері " + recipient.getEmail() + " үшін қабылдады");
    } catch (MailException ex) {
        return SendResult.failed("SMTP қатесі: " + ex.getMessage());
    }
}
```

Қалай жұмыс істейді

`AbstractDeliveryHandler` фабриканы конструкторда алады және тек интерфейстер арқылы жұмыс істейді: алдымен `createAvailabilityChecker().check()`, содан кейін `createFormatter().format()`, соңында `createSender().send()`. Демо режимде SMTP бапталмағандықтан e-mail отбасы «SMTP бапталмаған (демо режимі)» деп **шынайы** хабарлайды, ал жеткізу журналында «Жіберілмеді» күйі көрсетіледі — жүйе ешқашан жалған «жіберілді» деп жазбайды.

Неге осы паттерн

Жаңа арна (SMS, Telegram) қосу — үш өнімі бар бір жаңа фабрика жазу, клиент код өзгермейді. Отбасылар бір-бірімен араласпайтынына кепілдік береді.



3-сурет — Хабарландыру арналары: Abstract Factory отбасылары және Factory Method creator-лары

6.4. Observer (Бақылаушы)

Анықтамасы

Observer — бір объектінің (жариялаушының) күйі өзгергенде, оған тәуелді барлық объектілерге (бақылаушыларға) автоматты түрде хабар беретін мінез-құлықтық паттерн. Жариялаушы бақылаушыларды білмейді.

Шешетін мәселесі

Тауар есептен шығарылғанда: ескерту жасау, адамдарға хабарлау, журналға жазу және басқару панелін жаңарту керек. Егер `InventoryService` мұның бәрін тікелей шақырса, қойма модулі хабарландыруға, аудитке және интерфейске тәуелді болып қалар еді, ал әр жаңа реакция қойма кодын өзгертуді талап етер еді.

Жобадағы кластар

11-кесте — Observer қатысушылары

Рөлі	Класс	Не істейді
Оқиға (Subject хабары)	`LowStockEvent`, `AppointmentBookedEvent`, `FlightBookingConfirmedEvent`,	Не болғанын сипаттайды

Рөлі	Класс	Не істейді
	`DocumentApprovedEvent`, `DocumentRejectedEvent` т.б.	
Жариялаушы	`InventoryService`, `PatientAppointmentFacade`, `BookingEventAnnouncer`, `WorkflowEventRelay`	`ApplicationEventPublisher.publishEvent()`
Бақылаушы	`LowStockAlertRecorder`	Ескерту жазады (сол транзакцияда)
Бақылаушы	`NotificationEventListener`	Commit кейін хабарландыру жасайды
Бақылаушы	`DomainEventAuditListener`	Оқиғаны журналға жазады
Бақылаушы	`DashboardRefreshListener`	SSE арқылы басқару панелін жаңартады

13-листинг — Оқиға: төмен қор (backend/src/main/java/com/nexora/shared/event/LowStockEvent.java)

```
public record LowStockEvent(Long productId, String sku, String productName, int quantity, int
threshold,
                            int suggestedReorderQuantity, String replenishmentStrategy,
                            Long triggeredByUserId, Instant occurredAt) implements DomainEvent {
    @Override public String module() { return "INVENTORY"; }
    @Override public String summary() {
        return "Қор аз: " + productName + " (" + sku + "), қалдығы " + quantity + ", шеri " +
threshold;
    }
}
```

14-листинг — Жариялаушы: сыни деңгейден өткенде оқиға жариялау (backend/src/main/java/com/nexora/inventory/service/InventoryService.java)

```
private boolean evaluateStockLevel(Product product, int quantityBefore, Long userId) {
    LowStockDetectionStrategy detection = strategies.activeDetection();
    boolean nowCritical = detection.isCritical(product.getQuantity(), product.getMinStock());
    if (!nowCritical) {
        alerts.findByProductIdAndStatus(product.getId(), StockAlert.Status.OPEN)
                .forEach(a -> a.resolve(clock.instant()));
        alerts.findByProductIdAndStatus(product.getId(), StockAlert.Status.REQUEST_CREATED)
                .forEach(a -> a.resolve(clock.instant()));
        return false;
    }
    // One alert per critical episode: an open alert (or one with a pending request) suppresses
duplicates.
    boolean alreadyOpen = alerts.existsByProductIdAndStatusIn(product.getId(),
            EnumSet.of(StockAlert.Status.OPEN, StockAlert.Status.REQUEST_CREATED));
    if (alreadyOpen) {
        return false;
    }
    var replenishment = strategies.replenishment(product.getReplenishmentPolicy());
    events.publishEvent(new LowStockEvent(product.getId(), product.getSku(), product.getName(),
            product.getQuantity(), detection.criticalThreshold(product.getMinStock()),
            replenishment.suggestQuantity(product), product.getReplenishmentPolicy().name(), userId,
            clock.instant()));
    return true;
}
```

15-листинг — Бақылаушы 1: ескертуді сол транзакцияда жазу (backend/src/main/java/com/nexora/inventory/service/LowStockAlertRecorder.java)

```
@Component
public class LowStockAlertRecorder {


    private final StockAlertRepository alerts;
    private final ProductRepository products;


    public LowStockAlertRecorder(StockAlertRepository alerts, ProductRepository products) {
        this.alerts = alerts;
        this.products = products;
    }


    @EventListener
    public void onLowStock(LowStockEvent event) {
        alerts.save(new StockAlert(products.getReferenceById(event.productId()), event.quantity(),
event.threshold(),
                event.suggestedReorderQuantity(), event.replenishmentStrategy(), event.occurredAt()));
    }
}
```

16-листинг — Бақылаушы 2: commit-тен кейін хабарландыру жіберу (backend/src/main/java/com/nexora/notification/listener/NotificationEventListener.java)

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution = true)
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void on(LowStockEvent e) {
    Severity severity = e.quantity() == 0 ? Severity.CRITICAL : Severity.WARNING;
    dispatcher.dispatch(usersWithRoles(Role.ADMIN, Role.MANAGER, Role.EMPLOYEE), new NotificationDraft(
            NotificationCategory.INVENTORY, severity,
            (e.quantity() == 0 ? "Тауар таусылды: " : "Қор аз: ") + e.productName(),
            e.productName() + " (" + e.sku() + ") қалдығы: " + e.quantity() + ", шекті деңгей: " +
e.threshold()
                    + ". Ұсынылған тапсырыс: " + e.suggestedReorderQuantity() + " бірлік.",
            "/inventory?tab=alerts", "INVENTORY"));
}
```

17-листинг — Бақылаушы 3: басқару панелін нақты уақытта жаңарту (backend/src/main/java/com/nexora/dashboard/listener/DashboardRefreshListener.java)

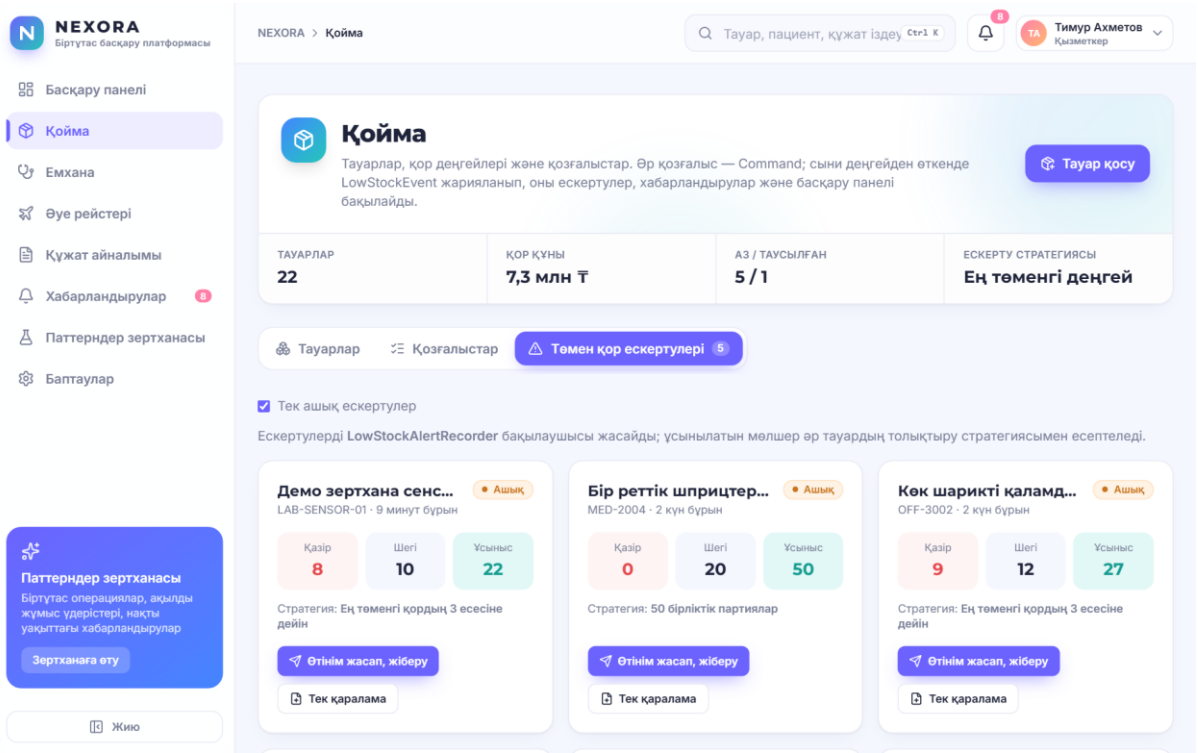
```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution = true)
public void on(DomainEvent event) {
    hub.broadcast("dashboard", Map.of("module", event.module(), "event",
event.getClass().getSimpleName(),
            "summary", event.summary(), "at", event.occurredAt().toString()));
}
```

Қалай жұмыс істейді

1. Қызметкер тауарды есептен шығарады → `AdjustInventoryCommand` → `InventoryService.adjustStock()`.
2. Қалдық сыни деңгейге жетсе, `LowStockEvent` жарияланады.
3. `LowStockAlertRecorder` (`@EventListener`) ескертуді **сол транзакцияда** сақтайды — қозғалыс кері қайтарылса, ескерту де жойылады.
4. Транзакция сәтті commit болғаннан кейін (`AFTER_COMMIT`) `NotificationEventListener` хабарландырулар жасайды, `DomainEventAuditListener` журналға жазады, `DashboardRefreshListener` барлық ашық панельдерге SSE-сигнал жібереді.

5. Браузерде тост пайда болады, қоңыраудағы сан артады, басқару панеліндегі көрсеткіш бетті қайта жүктемей жаңарады.

> **Сенімділік туралы шынайы түсіндірме**
> `AFTER_COMMIT` кері қайтарылған операция туралы ешкім хабар алмайтынына кепілдік береді. Бірақ Spring оқиғалары жадта өңделеді: commit пен өңдеуші арасында процесс тоқтаса, хабарландыру жоғалуы мүмкін. Кепілдендірілген жеткізу үшін transactional outbox қажет — ол жобада жоқ және бұл құжатталған.



4-сурет — Төмен қор ескертулері: Observer нәтижесі және толықтыру өтінімін жасау батырмалары

Тексеру

`ObserverUnitTest.lowStockEventCreatesAlertWithSuggestedQuantity` және интеграциялық `InventoryScenarioTest.writeOffBelowMinimumTriggersObserverChain` (ескерту жасалды, хабарландыру саны 1-ге артты, қайталап шығарғанда екінші ескерту жасалмайды).

6.5. State (Күй)

Анықтамасы

State — объектінің ішкі күйі өзгергенде оның мінез-құлқын өзгертуге мүмкіндік беретін мінез-құлықтық паттерн. Әр күй — жеке класс; ол қандай әрекеттерге рұқсат барын және келесі күйді өзі анықтайды.

Шешетін мәселесі

Паттерн болмаса, контроллерлерде `if (status == …)` тармақтары көбейеді және «клиент жіберген күйді жай жазып қою» қаупі туады. NEXORA-да клиент күйді жібере алмайды — тек әрекетті (`/confirm`, `/cancel`, `/approve`) сұрайды, ал рұқсат етілген-етілмегенін ағымдағы күй объектісі шешеді.

12-кесте — Брондаудың күйлері

Күй	Рұқсат етілген әрекеттер	Ереже
Күтуде (PENDING)	растау, болдырмау	Ұстау мерзімі (15 мин) өтсе — растауға болмайды; фондық тапсырма Мерзімі өткен күйге ауыстырады
Расталған (CONFIRMED)	болдырмау	Ұшуға 2 сағаттан аз қалса, болдырмауға болмайды
Болдырылмаған (CANCELLED)	—	Соңғы күй: қайта растауға болмайды
Мерзімі өткен (EXPIRED)	—	Соңғы күй

13-кесте — Құжаттың күйлері

Күй	Келесі күйлер
Қаралама (DRAFT)	бекітуге жіберу → Бекітуді күтуде (маршрут бос болса — бірден Бекітілген)
Бекітуді күтуде (PENDING_APPROVAL)	бекіту → Қаралуда немесе Бекітілген; қабылдамау → Қабылданбаған; қайтару → Қаралама
Қаралуда (IN_REVIEW)	бекіту (соңғы қадам) → Бекітілген; қабылдамау; қайтару
Бекітілген (APPROVED)	Соңғы күй — қаралама ретінде өңдеуге қайтаруға болмайды
Қабылданбаған (REJECTED)	Соңғы күй

18-листинг — Күй интерфейсі: әдепкі бойынша барлық әрекет тыйым салынған (backend/src/main/java/com/nexora/flight/state/BookingState.java)

```
public interface BookingState {

    BookingStatus status();

    /** Names of the operations a client may offer in this state (drives the UI buttons). */
    Set<String> allowedActions();

    default void confirm(FlightBooking booking, Instant now) {
        throw new InvalidStateTransitionException("booking", status().name(), "confirm");
    }

    default void cancel(FlightBooking booking, String reason, Instant now) {
        throw new InvalidStateTransitionException("booking", status().name(), "cancel");
    }

    default void expire(FlightBooking booking, Instant now) {
        throw new InvalidStateTransitionException("booking", status().name(), "expire");
```

```
    }

    /** True when seats held/booked by the booking must be released after leaving this state. */
    default boolean holdsSeats() {
        return false;
    }
}
```

19-листинг — Күтуде күйі: растау, болдырмау және мерзімнің өтуі (backend/src/main/java/com/nexora/flight/state/PendingBookingState.java)

```
@Component
public class PendingBookingState implements BookingState {

    @Override public BookingStatus status() { return BookingStatus.PENDING; }

    @Override public Set<String> allowedActions() { return Set.of("confirm", "cancel"); }

    @Override public boolean holdsSeats() { return true; }

    @Override
    public void confirm(FlightBooking booking, Instant now) {
        if (now.isAfter(booking.getExpiresAt())) {
            throw new BusinessRuleException("HOLD_EXPIRED", booking.getReference() + " брондауы бойынша
орындарды ұстау мерзімі өтті. "
                    + "Жаңа брондау жасау үшін қайта іздеңіз.");
        }
        booking.moveTo(BookingStatus.CONFIRMED, now, null);
    }

    @Override
    public void cancel(FlightBooking booking, String reason, Instant now) {
        booking.moveTo(BookingStatus.CANCELLED, now, reason);
    }

    @Override
    public void expire(FlightBooking booking, Instant now) {
        booking.moveTo(BookingStatus.EXPIRED, now, "Орындарды ұстау мерзімі растаудан бұрын аяқталды");
    }
}
```

20-листинг — Болдырылмаған күйі: соңғы күй, әрекеттер жоқ (backend/src/main/java/com/nexora/flight/state/CancelledBookingState.java)

```
@Component
public class CancelledBookingState implements BookingState {

    @Override public BookingStatus status() { return BookingStatus.CANCELLED; }

    @Override public Set<String> allowedActions() { return Set.of(); }
}
```

21-листинг — Күйді дерекқордағы мәннен қалпына келтіру (backend/src/main/java/com/nexora/flight/state/BookingStateRegistry.java)

```
public BookingStateRegistry(List<BookingState> all) {
    all.forEach(s -> states.put(s.status(), s));
    for (BookingStatus status : BookingStatus.values()) {
        if (!states.containsKey(status)) {
            throw new IllegalStateException("No BookingState implementation for " + status);
        }
    }
}

public BookingState of(FlightBooking booking) {
    return states.get(booking.getStatus());
```

```
}
```

22-листинг — Құжат: Қаралама күйінен жіберу (backend/src/main/java/com/nexora/workflow/state/DraftState.java)

```
public void submit(Document document, boolean routeHasSteps, Instant now) {
    if (routeHasSteps) {
        document.moveTo(DocumentStatus.PENDING_APPROVAL, now);
    } else {
        document.autoApprove(now);
    }
}
```

Қалай жұмыс істейді және қайта іске қосу

Дерекқорда күйдің атауы (enum) сақталады. Қосымша қайта іске қосылғанда `BookingStateRegistry` сол атау бойынша сәйкес күй объектісін қайтарады, сондықтан күй жоғалмайды. Рұқсат етілмеген әрекет `InvalidStateTransitionException` (HTTP 409) тудырады, мысалы: «Болдырылмаған» күйіндегі брондауды растау мүмкін емес. Интерфейстегі батырмалар да күй объектісінің `allowedActions()` тізімінен құрылады.



5-сурет — Паттерндер зертханасы: State тыйым салынған ауысуды қабылдамайды, Mediator журналы

Strategy-ден айырмашылығы

Strategy-ді **клиент немесе баптау** таңдайды, стратегиялар бір-бірін білмейді. State-те **күй объектісінің өзі** келесі күйді анықтайды және күйлер арасындағы ауысулар бизнес-ережелермен шектелген.

Тексеру

23-листинг — Тест: болдырылмаған брондауды қайта растауға болмайды (backend/src/test/java/com/nexora/patterns/BookingStatePatternTest.java)

```
void cancelledBookingCannotBeConfirmedAgain() {
    FlightBooking b = booking(now.plus(Duration.ofDays(2)));
    registry.of(b).cancel(b, "changed plans", now);
    assertThat(b.getStatus()).isEqualTo(BookingStatus.CANCELLED);
    assertThatThrownBy(() -> registry.of(b).confirm(b, now))
            .isInstanceOf(InvalidStateTransitionException.class)
            .hasMessageContaining("«Болдырылмаған» күйіндегі брондауды растау мүмкін емес");
    assertThat(registry.of(b).allowedActions()).isEmpty();
}
```

Сондай-ақ: `DocumentStatePatternTest` (6 тест), интеграциялық `FlightScenarioTest.cancelledBookingCannotBeConfirmed`, `WorkflowScenarioTest` (бекітілген құжатты қайта жіберу — 409).

6.6. Strategy (Стратегия)

Анықтамасы

Strategy — алгоритмдер отбасын анықтап, әрқайсысын жеке класқа инкапсуляциялайтын және оларды **жұмыс уақытында алмастыруға** мүмкіндік беретін мінез-құлықтық паттерн.

Жобадағы төрт стратегия отбасы

14-кесте — Strategy қолдану орындары

Отбасы	Нақты стратегиялар	Таңдалу тәсілі
Рейстерді сұрыптау (`FlightSortStrategy`)	`CheapestFirstStrategy` (price), `ShortestDurationStrategy` (duration), `EarliestDepartureStrategy` (departure)	API параметрі `sort` → кілт бойынша карта
Сыни қор деңгейі (`LowStockDetectionStrategy`)	`MinimumLevelStrategy` (≤ ең аз қор), `SafetyBufferStrategy` (≤ ең аз қор × 1,25)	`LOW_STOCK_POLICY` баптауы (Singleton арқылы)
Толықтыру мөлшері (`ReplenishmentStrategy`)	`TopUpToTargetStrategy` (3 есеге дейін), `FixedLotStrategy` (50-ден партия), `DemandBasedStrategy` (30 күндік сұраныс)	Әр тауардың толықтыру стратегиясы өрісі
Хабарландыру арнасы (`ChannelSelectionStrategy`)	`PreferenceBasedChannelStrategy`, `CriticalEscalationChannelStrategy`	Хабарламаның маңыздылығы

24-листинг — Стратегия интерфейсі (backend/src/main/java/com/nexora/flight/strategy/FlightSortStrategy.java)

```
public interface FlightSortStrategy {

    String key();
```

```
    String label();

    Comparator<Flight> comparator();
}
```

25-листинг — Нақты стратегия: ең арзан бірінші (backend/src/main/java/com/nexora/flight/strategy/CheapestFirstStrategy.java)

```
@Component
public class CheapestFirstStrategy implements FlightSortStrategy {
    @Override public String key() { return "price"; }
    @Override public String label() { return "Ең арзан баға"; }
    @Override public Comparator<Flight> comparator() {
        return Comparator.comparing(Flight::getBasePrice).thenComparing(Flight::getDepartureTime);
    }
}
```

26-листинг — Стратегияны if/else-сіз, карта арқылы таңдау (backend/src/main/java/com/nexora/flight/strategy/FlightSortStrategyResolver.java)

```
public FlightSortStrategyResolver(List<FlightSortStrategy> strategies) {
    this.byKey = strategies.stream().collect(Collectors.toMap(FlightSortStrategy::key,
Function.identity()));
}


public FlightSortStrategy resolve(String key) {
    FlightSortStrategy s = byKey.get(key == null || key.isBlank() ? DEFAULT : key);
    if (s == null) {
        throw new BusinessRuleException("UNKNOWN_SORT", "Белгісіз сұрыптау: '" + key + "'. Мүмкін
мәндер: " + byKey.keySet());
    }
    return s;
}
```

27-листинг — Стратегияны қолдану: іздеу нәтижесін сұрыптау (backend/src/main/java/com/nexora/flight/facade/BookingFacade.java)

```
FlightSortStrategy strategy = sortStrategies.resolve(sort);
Airport origin = airports.findByCodeIgnoreCase(originCode).orElseThrow(() -> new
NotFoundException("Airport", originCode));
Airport destination = airports.findByCodeIgnoreCase(destinationCode)
        .orElseThrow(() -> new NotFoundException("Airport", destinationCode));
ZoneId zone = origin.zone();
Instant from = date.atStartOfDay(zone).toInstant();
Instant to = date.plusDays(1).atStartOfDay(zone).toInstant();
List<FlightDto> result = flights.search(origin.getId(), destination.getId(), from, to, clock.instant(),
passengers)
        .stream().sorted(strategy.comparator()).map(FlightMapper::toDto).toList();
```

28-листинг — Сұранысқа негізделген толықтыру стратегиясы (backend/src/main/java/com/nexora/inventory/strategy/DemandBasedStrategy.java)

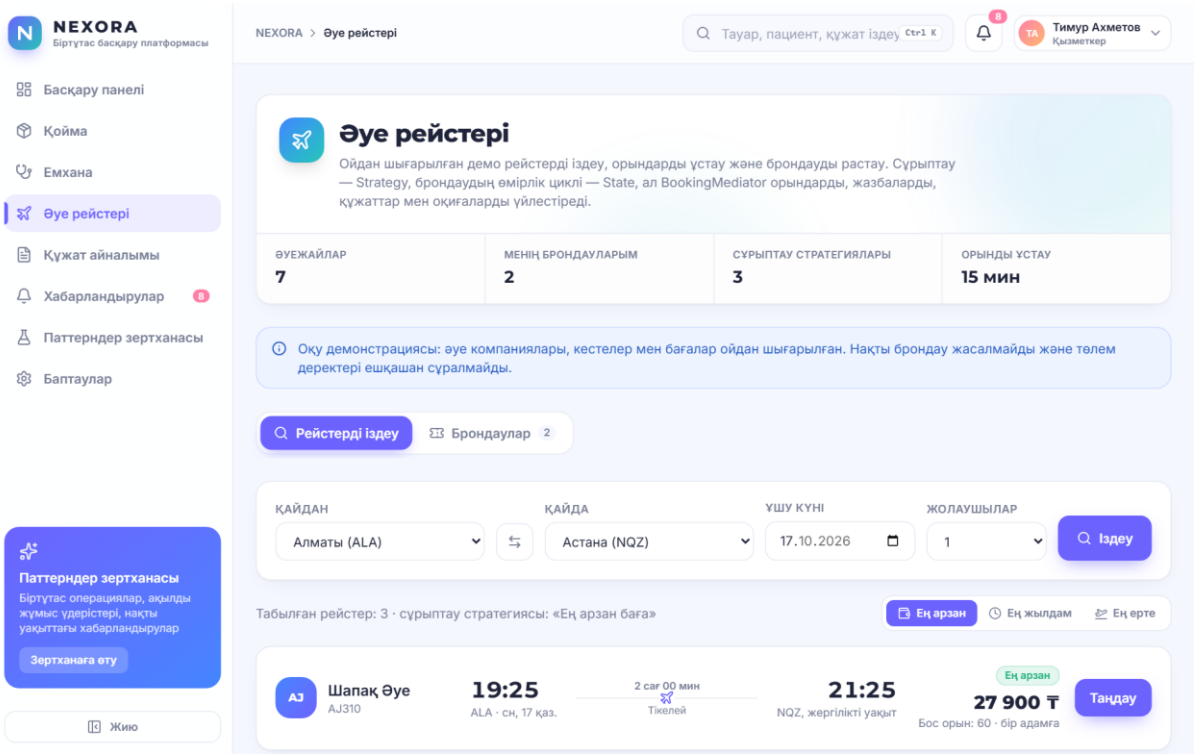
```
@Override public ReplenishmentPolicy policy() { return ReplenishmentPolicy.DEMAND_BASED; }


@Override public String description() { return "30 күндік сұраныс × 21 күн + қауіпсіздік қоры"; }


@Override
public int suggestQuantity(Product product) {
    long outflow = transactions.outflowSince(product.getId(),
clock.instant().minus(Duration.ofDays(LOOKBACK_DAYS)));
    double daily = outflow / (double) LOOKBACK_DAYS;
    int coverage = (int) Math.ceil(daily * COVERAGE_DAYS);
    return Math.max(coverage + product.getMinStock() - product.getQuantity(), 1);
}
```

Нәтижеге әсері

Стратегиялар API нәтижесін нақты өзгертеді: бір іздеуде «Ең арзан» бойынша AJ310 (27 900 ₸) бірінші тұрады, ал «Ең жылдам» бойынша SW205 (1 сағ 35 мин). Толықтыру стратегиясы ескертудегі «ұсынылған тапсырыс» санын есептейді, ал арна стратегиясы сыни хабарламаны пайдаланушы өшіріп қойса да «қосымша ішінде» арнасына міндетті түрде жібереді.



6-сурет — Рейстерді іздеу: «Ең арзан / Ең жылдам / Ең ерте» стратегиялары

Тексеру

`StrategyPatternTest`: `flightSortStrategiesProduceDifferentOrders`, `replenishmentStrategiesComputeDifferentQuantities` (55 / 50 / 78 бірлік), `lowStockDetectionStrategiesDiffer`, `channelStrategyRespectsPreferencesUnlessCritical`; интеграциялық `FlightScenarioTest.searchSortedByPriceAndByDuration`.

6.7. Facade (Қасбет)

Анықтамасы

Facade — күрделі ішкі жүйеге **бір қарапайым кіру нүктесін** беретін құрылымдық паттерн.

Шешетін мәселесі

Қабылдауға жазылу үшін: пациентті табу, қол жеткізу құқығын тексеру, дәрігер жолын құлыптау, жұмыс уақытын тексеру, дәрігер мен пациент үшін қабаттасуды тексеру, жазбаны сақтау, оқиға жариялау керек. Бұл қадамдардың **реті**

маңызды (алдымен құлыптау, кейін тексеру). Контроллер мұны өзі ұйымдастырса, логика шашырап кетер еді.

Жобадағы фасадтар

- `PatientAppointmentFacade` — `book`, `reschedule`, `cancel`, `changeStatus`.
- `BookingFacade` — `search`, `seatMap`, `create`, `confirm`, `cancel`, `expire`, `history`.
- `DocumentWorkflowFacade` — `create`, `updateDraft`, `submit`, `approve`, `reject`, `returnForRevision`, `comment`, `replenishmentFromAlert`, `issueSystemDocument`.

29-листинг — PatientAppointmentFacade.book — бір шақыру, жеті қадам (backend/src/main/java/com/nexora/clinic/facade/PatientAppointmentFacade.java)

```
public AppointmentDto book(BookAppointmentRequest request, AuthenticatedUser actor) {
    Patient patient = patients.findById(request.patientId())
            .orElseThrow(() -> new NotFoundException("Patient", request.patientId()));
    access.checkCanBookFor(actor, patient);


    Doctor doctor = doctors.lockById(request.doctorId())
            .orElseThrow(() -> new NotFoundException("Doctor", request.doctorId()));
    if (!doctor.isActive()) {
        throw new BusinessRuleException("DOCTOR_INACTIVE", "Бұл дәрігер қазір қабылдау жүргізбейді");
    }
    Instant start = request.startTime();
    Instant end = start.plus(schedule.slotLength());
    schedule.validateWithinWorkingHours(doctor, start);
    schedule.ensureNoConflicts(doctor, patient.getId(), start, end, null);


    Appointment saved = appointments.save(new Appointment(patient, doctor, start, end,
request.reason().trim(), actor.id()));
    events.publishEvent(new AppointmentBookedEvent(saved.getId(), patient.fullName(),
patient.getUserId(),
            doctor.getFullName(), doctor.getUserId(), start, actor.id(), clock.instant()));
    return ClinicMapper.toDto(saved);
}
```

30-листинг — Контроллер фасадты бір рет шақырады (Command арқылы) (backend/src/main/java/com/nexora/clinic/controller/ClinicController.java)

```
public AppointmentDto book(@Valid @RequestBody BookAppointmentRequest request) {
    AuthenticatedUser user = CurrentUser.require();
    return invoker.run(SimpleCommand.of("BOOK_APPOINTMENT", "Appointment", () -> facade.book(request,
user),
            AppointmentDto::id, "дәрігер " + request.doctorId() + ", пациент " + request.patientId() +
", уақыты " + request.startTime()));
}
```

Неге осы паттерн

Фасадтар шағын және әр ішкі жүйеге бөлек — бір «құдіретті» класс жасалмаған. Қадамдардың дұрыс реті бір жерде сақталады, контроллер тек бір әдісті шақырады.

> **Facade мен Mediator айырмашылығы (жиі қойылатын сұрақ)**
> **Facade** — сыртқы клиентке (контроллерге) арналған жеңілдетілген интерфейс; ішкі жүйе фасадты білмейді. **Mediator** — ішкі жүйенің **ішіндегі** өзара әрекеттесу орталығы; әріптестер медиаторды біледі және оған сигнал жібереді, ал медиатор келесі кім әрекет ететінін шешеді. NEXORA-да `BookingFacade` → `BookingMediator`-ды шақырады: фасад — «кіру есігі», медиатор — «диспетчер».

Тексеру

`ClinicScenarioTest` (бос уақытқа жазылу, қабаттасуды қабылдамау, жұмыс уақытынан тыс, 4 параллель сұраныс), `FlightScenarioTest` (растау құжат шығарады, болдырмау орындарды босатады).

6.8. Mediator (Делдал)

Анықтамасы

Mediator — объектілердің (әріптестердің) бір-бірімен тікелей байланысуын болдырмай, олардың өзара әрекеттесуін бір делдал объектіге шоғырландыратын мінез-құлықтық паттерн.

Шешетін мәселесі

Брондау сценарийінде алты компонент қатысады: рейсті тексеру, орындарды бөлу, брондауды жазу, күйді өзгерту, растау құжатын шығару, оқиға жариялау. Егер олар бір-бірін тікелей шақырса, тәуелділіктер «өрмекшінің торына» айналар еді, ал сценарийді өзгерту барлық кластарды өзгертуді талап етер еді.

15-кесте — Брондау медиаторының әріптестері

Әріптес	Жалғыз міндеті
`FlightAvailabilityChecker`	Рейс жолын құлыптап, брондауға болатынын тексеру
`SeatAllocator`	Орындарды ұстау, брондау және босату
`BookingRecorder`	Брондауды бірегей кодпен сақтау
`BookingStateKeeper`	State объектісі арқылы күйді өзгерту
`ConfirmationIssuer`	Растау құжатын шығару (workflow порты арқылы)
`BookingEventAnnouncer`	Observer оқиғасын жариялау

31-листинг — Әріптестің базалық класы: тек медиаторға хабарлайды (backend/src/main/java/com/nexora/flight/mediator/BookingColleague.java)

```
public abstract class BookingColleague {

    private BookingMediator mediator;

    void attach(BookingMediator mediator) {
        this.mediator = mediator;
    }

    protected void report(BookingSignal signal, BookingContext ctx) {
        mediator.notify(this, signal, ctx);
    }

    public abstract String role();
}
```

32-листинг — Медиатор: әріптестерді тіркеу (Spring циклінсіз) (backend/src/main/java/com/nexora/flight/mediator/FlightBookingMediator.java)

```
public FlightBookingMediator(FlightAvailabilityChecker checker, SeatAllocator seats, BookingRecorder
recorder,
                             BookingStateKeeper stateKeeper, ConfirmationIssuer issuer,
                             BookingEventAnnouncer announcer) {
    this.checker = checker;
    this.seats = seats;
    this.recorder = recorder;
    this.stateKeeper = stateKeeper;
    this.issuer = issuer;
    this.announcer = announcer;
    for (BookingColleague c : new BookingColleague[]{checker, seats, recorder, stateKeeper, issuer,
announcer}) {
        c.attach(this);
    }
}
```

33-листинг — Өзара әрекеттесу ережелері бір жерде (backend/src/main/java/com/nexora/flight/mediator/FlightBookingMediator.java)

```
public void notify(BookingColleague sender, BookingSignal signal, BookingContext ctx) {
    ctx.log("Mediator <- " + (sender == null ? "BookingFacade" : sender.role()) + ": " + signal);
    switch (signal) {
        case CREATE_REQUESTED -> checker.verify(ctx);
        case FLIGHT_VERIFIED -> seats.hold(ctx);
        case SEATS_HELD -> recorder.record(ctx);
        case BOOKING_RECORDED -> announcer.announceCreated(ctx);

        case CONFIRM_REQUESTED -> stateKeeper.confirm(ctx);
        case STATE_CONFIRMED -> seats.book(ctx);
        case SEATS_BOOKED -> issuer.issue(ctx);
        case DOCUMENT_ISSUED -> announcer.announceConfirmed(ctx);

        case CANCEL_REQUESTED -> stateKeeper.cancel(ctx);
        case EXPIRE_REQUESTED -> stateKeeper.expire(ctx);
        case STATE_CLOSED -> seats.release(ctx);
        case SEATS_RELEASED -> announcer.announceClosed(ctx);

        case ANNOUNCED -> ctx.log("Mediator: сценарий аяқталды");
    }
}
```

34-листинг — Әріптес өз жұмысын бітіріп, медиаторға сигнал жібереді (backend/src/main/java/com/nexora/flight/mediator/SeatAllocator.java)

```
chosen.forEach(FlightSeat::hold);
ctx.flight.reserveSeats(chosen.size());
ctx.seats = chosen;
ctx.log(role() + ": ұсталған орындар " + chosen.stream().map(FlightSeat::getSeatNumber).toList());
report(BookingSignal.SEATS_HELD, ctx);
```

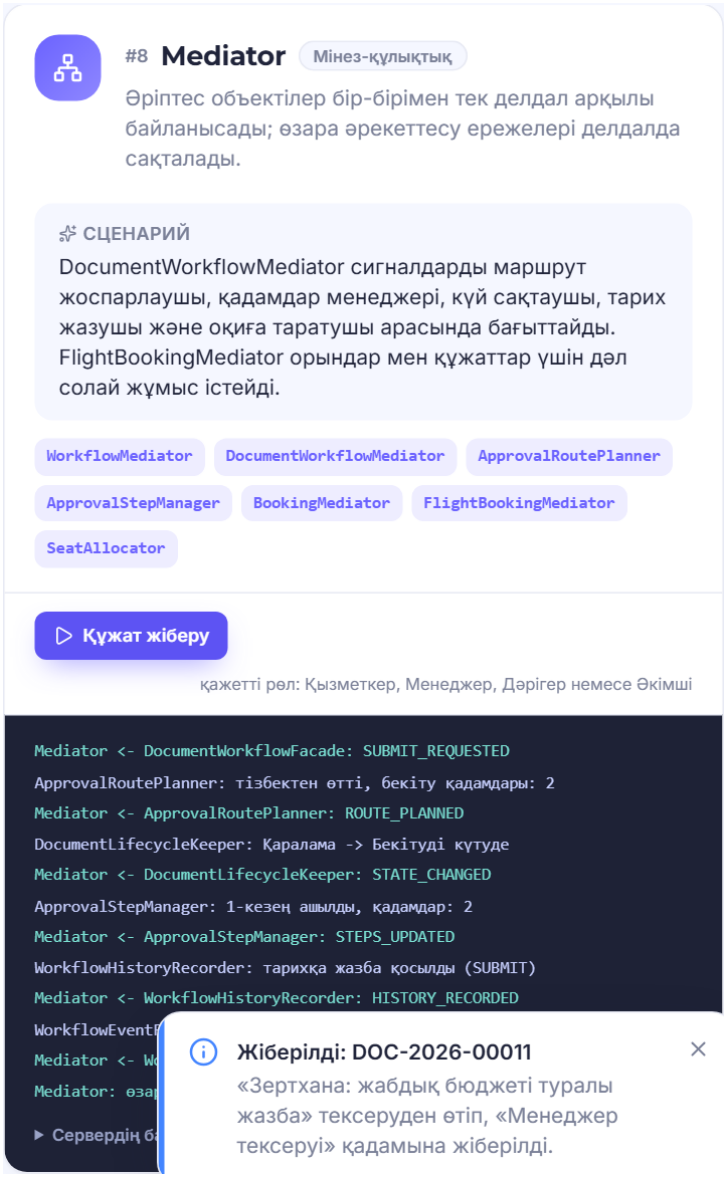
35-листинг — Құжат айналымының медиаторы (backend/src/main/java/com/nexora/workflow/mediator/DocumentWorkflowMediator.java)

```
public void notify(WorkflowColleague sender, WorkflowSignal signal, WorkflowContext ctx) {
    ctx.log("Mediator <- " + (sender == null ? "DocumentWorkflowFacade" : sender.role()) + ": " +
signal);
    boolean submit = ctx.action == WorkflowAction.SUBMIT;
    switch (signal) {
        case SUBMIT_REQUESTED -> planner.plan(ctx);
        case ROUTE_PLANNED -> lifecycle.apply(ctx);
        case DECISION_REQUESTED -> stepManager.decide(ctx);
        case STATE_CHANGED -> {
            if (submit && !ctx.review.steps().isEmpty()) {
                stepManager.openRound(ctx);
            } else {
                history.record(ctx);
            }
```

```
        }
        case STEPS_UPDATED -> {
            if (submit) {
                history.record(ctx);
            } else {
                lifecycle.apply(ctx);
            }
        }
        case HISTORY_RECORDED -> relay.publish(ctx);
        case PUBLISHED -> ctx.log("Mediator: өзара әрекет аяқталды");
    }
}
```

Қалай жұмыс істейді

Әр әріптес өз ісін бітіріп, `report(signal)` арқылы медиаторға хабарлайды; медиатор `switch` арқылы келесі әріптесті шақырады. Брондау: CREATE_REQUESTED → тексеру → FLIGHT_VERIFIED → орындар → SEATS_HELD → жазу → BOOKING_RECORDED → оқиға. Растау: CONFIRM_REQUESTED → State → STATE_CONFIRMED → орындар → SEATS_BOOKED → құжат → DOCUMENT_ISSUED → оқиға. Сценарийді өзгерту (мысалы, алаяқтықты тексеру қадамын қосу) тек медиаторды өзгертуді талап етеді. Әр жауапта `coordinationLog` журналы қайтарылады, ол интерфейсте көрсетіледі.



7-сурет — WorkflowMediator үйлестіру журналы (Паттерндер зертханасы)

Тексеру

`FlightBookingMediatorTest`: `creationSignalsAreRoutedInOrder`, `confirmationIssuesDocumentOnlyAfterSeatsAreBooked`, `cancellationReleasesSeatsThenAnnounces`.

6.9. Chain of Responsibility (Жауапкершілік тізбегі)

Анықтамасы

Chain of Responsibility — сұранысты өңдеушілер тізбегі бойымен жіберетін мінез-құлықтық паттерн: әр өңдеуші өз тексерісін жасап, сұранысты әрі қарай береді немесе тізбекті тоқтатады.

Шешетін мәселесі

Құжатты бекітуге жібермес бұрын бірнеше тәуелсіз тексеру керек, ал бекіту маршруты құжат түріне және сомасына байланысты. Барлық ережені бір әдіске жазу

— үлкен, өзгертуге қиын `if` блогы. Тізбек әр ережені бөлек класқа бөледі және жаңа ережені басқа кодты өзгертпей қосуға мүмкіндік береді.

16-кесте — Тізбектің өңдеушілері (@Order ретімен)

Рет	Өңдеуші	Тексеріс / әрекет
10	`RequiredFieldsHandler`	Міндетті өрістер (тақырып, мазмұн, сома, байланысты нысан)
20	`DataIntegrityHandler`	Сома ауқымы, ондық белгілер, мазмұн ұзындығы, нысан түрі
30	`AuthorPermissionHandler`	Автордың рөлі осы түрдегі құжатты жасай ала ма
40	`ManagerApprovalHandler`	«Менеджер тексеруі» қадамын қосу (әрқашан немесе 250 000 ₸-дан жоғары)
50	`AdditionalApprovalHandler`	«Соңғы бекіту» (Әкімші) — 1 500 000 ₸-дан жоғары немесе медициналық есеп
100	`RoutingDecisionHandler`	Соңғы шешім: маршрут немесе автобекіту; «төрт көз» қағидаты

36-листинг — Абстрактілі өңдеуші: тексеріп, келесісіне беру немесе тоқтату (backend/src/main/java/com/nexora/workflow/chain/DocumentCheckHandler.java)

```
public abstract class DocumentCheckHandler {

    private DocumentCheckHandler next;

    final DocumentCheckHandler linkWith(DocumentCheckHandler next) {
        this.next = next;
        return next;
    }

    public final void handle(DocumentReview review) {
        check(review);
        if (!review.stopped() && next != null) {
            next.handle(review);
        }
    }

    /** Perform the check; call {@code review.stop(...)} to break the chain. */
    protected abstract void check(DocumentReview review);

    public abstract String name();

    protected void pass(DocumentReview review, String message) {
        review.record(name(), DocumentReview.Outcome.PASSED, message);
    }

    protected void fail(DocumentReview review, String reason) {
        review.stop(name(), reason);
    }
}
```

37-листинг — Нақты өңдеуші: міндетті өрістер (backend/src/main/java/com/nexora/workflow/chain/RequiredFieldsHandler.java)

```
@Component
@Order(10)
public class RequiredFieldsHandler extends DocumentCheckHandler {
```

```
    @Override public String name() { return "RequiredFieldsHandler"; }

    @Override
    protected void check(DocumentReview r) {
        if (r.title() == null || r.title().isBlank()) {
            fail(r, "Тақырып міндетті");
            return;
        }
        if (r.content() == null || r.content().isBlank()) {
            fail(r, "Мазмұны міндетті");
            return;
        }
        if (r.type().amountRequired() && r.amount() == null) {
            fail(r, "«" + r.type().label() + "» үшін сома міндетті");
            return;
        }
        if (r.type().relatedEntityType() != null && r.relatedEntityId() == null) {
            fail(r, "«" + r.type().label() + "» байланысты нысанға сілтеме жасауы керек (" +
relatedLabel(r.type().relatedEntityType()) + ")");
            return;
        }
        pass(r, "Барлық міндетті өрістер толтырылған");
    }
```

38-листинг — Нақты өңдеуші: қосымша (соңғы) бекіту қадамы (backend/src/main/java/com/nexora/workflow/chain/AdditionalApprovalHandler.java)

```
protected void check(DocumentReview r) {
    if (r.amount() != null && r.amount().compareTo(settings.additionalApprovalThreshold()) > 0) {
        r.addStep(name(), new DocumentReview.PlannedStep(STEP, Role.ADMIN,
                "сома " + com.nexora.shared.i18n.Kk.money(settings.additionalApprovalThreshold()) + "
шегінен жоғары"));
        return;
    }
    if (r.type() == DocumentType.MEDICAL_APPOINTMENT_REPORT) {
        r.addStep(name(), new DocumentReview.PlannedStep(STEP, Role.ADMIN,
                "медициналық есепке емхана әкімшісі қосымша қол қояды"));
        return;
    }
    r.record(name(), DocumentReview.Outcome.SKIPPED, "Қосымша бекіту қажет емес");
}
```

39-листинг — Тізбекті Spring бинтерінен автоматты құрастыру (backend/src/main/java/com/nexora/workflow/chain/DocumentProcessingChain.java)

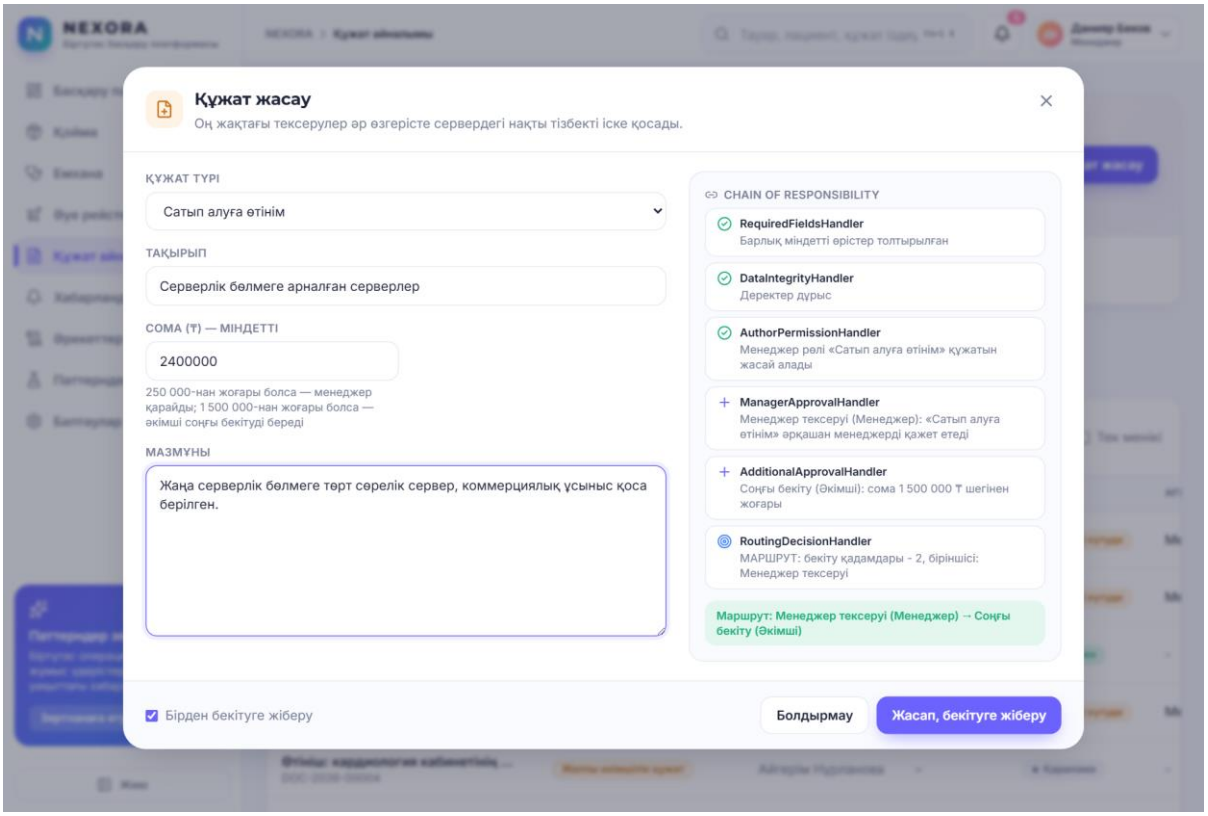
```
public DocumentProcessingChain(List<DocumentCheckHandler> handlers) {
    if (handlers.isEmpty()) {
        throw new IllegalStateException("Document chain needs at least one handler");
    }
    this.head = handlers.getFirst();
    DocumentCheckHandler current = head;
    for (int i = 1; i < handlers.size(); i++) {
        current = current.linkWith(handlers.get(i));
    }
    this.handlerNames = handlers.stream().map(DocumentCheckHandler::name).toList();
}

public DocumentReview review(DocumentReview review) {
    head.handle(review);
    return review;
}
```

Қалай жұмыс істейді

Құжат жасау терезесінде әр өзгерісте `POST /api/workflows/validate` арқылы тізбек «құрғақ» іске қосылады, ал оң жақта қай өңдеуші өткені, қайсысы қадам қосқаны және тізбек қай жерде тоқтағаны көрсетіледі. Бекітуге жібергенде

`ApprovalRoutePlanner` (Mediator әріптесі) сол тізбекті іске қосады; тізбек тоқтаса, құжат жіберілмейді.



8-сурет — Chain of Responsibility-дің нақты уақыттағы алдын ала қарауы (2 400 000 ₸ сатып алу өтінімі)

Кеңейтілуі

Жаңа ереже қосу — `@Order` бар бір жаңа `@Component` жазу. Тізбек оны автоматты түрде тиісті орынға қояды. Бұл тестпен дәлелденген:

40-листинг — Тест: жаңа өңдеуші басқа кодты өзгертпей қосылады (backend/src/test/java/com/nexora/patterns/ChainOfResponsibilityTest.java)

```
void newHandlerIsAddedWithoutChangingOthers() {
    DocumentCheckHandler noWeekendWork = new DocumentCheckHandler() {
        @Override public String name() { return "ComplianceHandler"; }
        @Override protected void check(DocumentReview r) {
            if (r.title().contains("forbidden")) {
                fail(r, "Compliance rule violated");
            } else {
                pass(r, "Compliant");
            }
        }
    };
    DocumentReview r = chain(noWeekendWork).review(new
DocumentReview(DocumentType.GENERAL_ADMINISTRATIVE, "forbidden title",
            "Some long enough content", null, null, null, Role.EMPLOYEE, false));
    assertThat(r.stopped()).isTrue();
    assertThat(r.trace().getLast().handler()).isEqualTo("ComplianceHandler");
}
```

6.10. Command (Команда)

Анықтамасы

Command — сұранысты (әрекетті) жеке объектіге айналдыратын мінез-құлықтық паттерн. Мұндай объектіні беруге, кейінге қалдыруға, журналға жазуға және бірыңғай түрде орындауға болады.

Шешетін мәселесі

Пайдаланушының әр маңызды әрекеті (қор қозғалысы, құжатты бекіту, брондауды болдырмау) транзакцияда орындалып, нәтижесі — сәтті ме, сәтсіз бе — аудит журналына жазылуы керек. Мұны әр контроллерде қайталау — қателерге әкеледі. Command барлық әрекеттерді бір пішінге келтіреді, ал `CommandInvoker` ортақ мінез-құлықты бір рет қосады.

17-кесте — Command қатысушылары

Рөлі	Класс
Command интерфейсі	`shared/command/Command<R>` — `name`, `entityType`, `execute`, `entityId`, `details`
Invoker (орындаушы)	`CommandInvoker` — транзакция + аудит (сәтсіздік те жазылады)
Receiver (қабылдаушы)	`InventoryService`, `BookingFacade`, `DocumentWorkflowFacade`, `PatientAppointmentFacade`
Нақты командалар	`AdjustInventoryCommand`, `CreateProductCommand`, `UpdateProductCommand`, `CreateDocumentCommand`, `SubmitDocumentCommand`, `ApproveDocumentCommand`, `RejectDocumentCommand`, `ReturnDocumentCommand`, `CreateFlightBookingCommand`, `ConfirmFlightBookingCommand`, `CancelFlightBookingCommand`, `SimpleCommand`

41-листинг — Command интерфейсі (backend/src/main/java/com/nexora/shared/command/Command.java)

```
public interface Command<R> {

    /** Stable action name written to the audit log, e.g. {@code APPROVE_DOCUMENT}. */
    String name();

    /** Entity type the command acts on, e.g. {@code Document}. */
    String entityType();

    /** Performs the action against the receiver. Runs inside the invoker's transaction. */
    R execute();

    /** Id of the affected entity, available after execution (creation commands learn it from the
result). */
    default String entityId(R result) {
        return null;
    }

    /** Human readable details for the audit log. */
    default String details() {
        return "";
```

```
    }
}
```

42-листинг — Invoker: транзакция және аудит (сәтсіздік те журналға түседі) (backend/src/main/java/com/nexora/shared/command/CommandInvoker.java)

```
public <R> R run(Command<R> command) {
    AuthenticatedUser actor = CurrentUser.find().orElse(null);
    Long actorId = actor == null ? null : actor.id();
    String actorName = actor == null ? "system" : actor.getUsername();
    try {
        R result = transactionTemplate.execute(status -> command.execute());
        auditTrail.record(actorId, actorName, command.name(), command.entityType(),
                command.entityId(result), AuditTrail.Outcome.SUCCESS, command.details());
        return result;
    } catch (RuntimeException ex) {
        log.debug("Command {} failed: {}", command.name(), ex.getMessage());
        auditTrail.record(actorId, actorName, command.name(), command.entityType(),
                command.entityId(null), AuditTrail.Outcome.FAILURE, truncate(command.details() + " | "
+ ex.getMessage()));
        throw ex;
    }
}
```

43-листинг — Нақты команда: қор қозғалысы (backend/src/main/java/com/nexora/inventory/command/AdjustInventoryCommand.java)

```
public final class AdjustInventoryCommand implements Command<AdjustmentResult> {


    private final InventoryService receiver;
    private final Long productId;
    private final AdjustStockRequest request;
    private final Long userId;


    public AdjustInventoryCommand(InventoryService receiver, Long productId, AdjustStockRequest
request, Long userId) {
        this.receiver = receiver;
        this.productId = productId;
        this.request = request;
        this.userId = userId;
    }


    @Override public String name() { return "ADJUST_INVENTORY"; }
    @Override public String entityType() { return "Product"; }
    @Override public AdjustmentResult execute() { return receiver.adjustStock(productId, request,
userId); }
    @Override public String entityId(AdjustmentResult result) { return String.valueOf(productId); }


    @Override
    public String details() {
        return request.type() + " " + request.quantity() + " (" + request.reason() + ")";
    }
}
```

44-листинг — Контроллер команданы жасап, Invoker-ге береді (backend/src/main/java/com/nexora/inventory/controller/InventoryController.java)

```
@PostMapping("/products/{id}/adjustments")
public AdjustmentResult adjust(@PathVariable Long id, @Valid @RequestBody AdjustStockRequest request) {
    return invoker.run(new AdjustInventoryCommand(inventory, id, request, CurrentUser.require().id()));
}
```

45-листинг — Аудит бөлек транзакцияда (кері қайтарылған әрекет те тіркеледі) (backend/src/main/java/com/nexora/audit/service/AuditService.java)

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

```
public void record(Long actorId, String actorUsername, String action, String entityType, String
entityId,
                   Outcome outcome, String details) {
    String safeDetails = details == null ? null : (details.length() > 1000 ? details.substring(0, 1000)
: details);
    repository.save(new AuditLog(actorId, actorUsername, action, entityType, entityId, outcome.name(),
            safeDetails, clock.instant()));
}
```

Undo туралы

Undo **әдейі жасалмаған**: құжатты бекітуді, жіберілген хабарландыруды немесе босатылған орынды бизнес-ережелер бойынша қауіпсіз «кері қайтару» мүмкін емес. Жалған undo жасау пайдаланушыны адастырар еді.

Тексеру

`CommandInvokerTest`: `successfulCommandCommitsAndIsAudited`, `failingCommandRollsBackAndIsAuditedAsFailure`. «Әрекеттер журналы» бетінде әр команданың аты, орындаушысы, нысаны және нәтижесі (Сәтті/Сәтсіз) көрінеді.

6.11. Ұқсас паттерндерді салыстыру

18-кесте — Жиі шатастырылатын паттерн жұптары

Жұп	Айырмашылығы	NEXORA-дағы мысал
Factory Method vs Abstract Factory	FM — мұрагерлік арқылы бір өнім; AF — композиция арқылы үйлесімді өнімдер отбасы	`DeliveryHandlerCreator` vs `NotificationChannelFactory`
Facade vs Mediator	Facade — сыртқа арналған қарапайым интерфейс; Mediator — ішкі әріптестерді үйлестіреді	`BookingFacade` → `FlightBookingMediator`
State vs Strategy	State-те күй келесі күйді өзі анықтайды; Strategy-ді клиент/баптау таңдайды	`PendingBookingState` vs `CheapestFirstStrategy`
Observer vs Mediator	Observer — бір-біріне байланыссыз бақылаушылар; Mediator — белгілі ретпен үйлестіру	`NotificationEventListener` vs `DocumentWorkflowMediator`
Chain of Responsibility vs Command	Chain — сұранысты кімнің өңдейтінін анықтайды; Command — әрекетті объект ретінде инкапсуляциялайды	`DocumentCheckHandler` vs `SubmitDocumentCommand`

7. Қауіпсіздік

- **Бір сеанс схемасы:** серверлік HTTP-сеанс (`NEXORA_SESSION`, HttpOnly, SameSite=Lax); JWT араласпайды.
- **CSRF:** double-submit cookie (`XSRF-TOKEN` + `X-XSRF-TOKEN` тақырыбы); өзгертетін сұраныстар токенсіз 403 алады.
- **Кіру:** BCrypt; кіргеннен кейін сеанс идентификаторы мен CSRF токені ауыстырылады (session fixation-нан қорғау); 5 сәтсіз әрекеттен кейін 5 минутқа бұғаттау (HTTP 429).
- **Авторизация серверде:** `@PreAuthorize` рөлдері + нысан деңгейіндегі тексерулер (`ClinicAccessPolicy`, `DocumentAccessPolicy`, брондау иесі); бөтен нысан 404 қайтарады.
- **«Төрт көз» қағидаты:** автор өз құжатын бекіте алмайды; Менеджер Әкімшінің соңғы бекітуін жасай алмайды.
- **Валидация:** барлық DTO-да Bean Validation (қазақша хабарламалар), Zod арқылы формаларда; бірыңғай қате пішімі stack trace-сіз.
- **Құпиялар:** тек орта айнымалыларында (`.env` Git-ке кірмейді); нақты пациент деректері мен төлем деректері жоқ.

7.1. Авторизация қатесін табу және түзету

Сынақ кезінде браузерде ешбір демо-аккаунт кіре алмайтыны анықталды, ал API (curl) барлық аккаунтты қабылдады. Себебі фронтендте болды: сәтті кіруден кейін `queryClient.clear()` шақырылып, `AuthProvider` жазылған `['me']` сұранысы кэштен өшірілді, сондықтан пайдаланушы деректері провайдерге жетпей, жүйе қайта кіру бетіне бағыттады. Түзету — тек басқа деректерді тазалап, `['me']` сұранысын сақтау. Қауіпсіздік (Spring Security, CSRF) өшірілмеді.

46-листинг — Түзетілген кіру/шығу логикасы (frontend/src/auth/AuthContext.tsx)

```
const resetUserData = useCallback((profile: UserProfile | null) => {
  queryClient.removeQueries({ predicate: (q) => q.queryKey[0] !== 'me' })
  queryClient.setQueryData(['me'], profile)
}, [queryClient])


const login = useCallback(async (username: string, password: string) => {
  await ensureCsrf()
  const profile = await api.post<UserProfile>('/auth/login', { username, password })
  await ensureCsrf() // the token is rotated on login
  resetUserData(profile)
  return profile
}, [resetUserData])
```

8. Тестілеу

Барлық тесттер нақты іске қосылды. Backend тесттері H2 дерекқорында (MySQL режимі) **сол Flyway миграцияларымен** жұмыс істейді, яғни нақты сұлба мен демо деректерде тексеріледі.

19-кесте — Backend тест кластары

Тест класы	Тест саны	Не тексереді
BookingStatePatternTest	5	State: рұқсат етілген және тыйым салынған ауысулар
DocumentStatePatternTest	6	State: құжаттың өмірлік циклі
StrategyPatternTest	4	Strategy: әртүрлі нәтижелер
ChainOfResponsibilityTest	6	Тізбек: тоқтау, қадам қосу, кеңейту
NotificationFactoriesTest	4	Factory Method және Abstract Factory
CommandInvokerTest	2	Command: транзакция және аудит
FlightBookingMediatorTest	3	Mediator: сигналдар реті
ObserverUnitTest	1	Observer: ескерту жасау
InventoryScenarioTest	4	Тауар жасау, сыни деңгейге дейін шығару, хабарландыру
ClinicScenarioTest	4	Жазылу, қабаттасу, 4 параллель сұраныс
FlightScenarioTest	5	Сұрыптау, растау/болдырмау, тыйым салынған ауысу
WorkflowScenarioTest	4	Құжат жасау, бекіту, себеппен қабылдамау
SecurityIntegrationTest	7	Кіру, CSRF, рөлдерге тыйым
PlatformIntegrationTest	4	Singleton, басқару панелі, зертхана
LocalizationAndDemoAccountsTest	4	Қазақша деректер мен хабарламалар, аккаунттарды қалпына келтіру
Барлығы	63	0 қате, 0 сәтсіздік

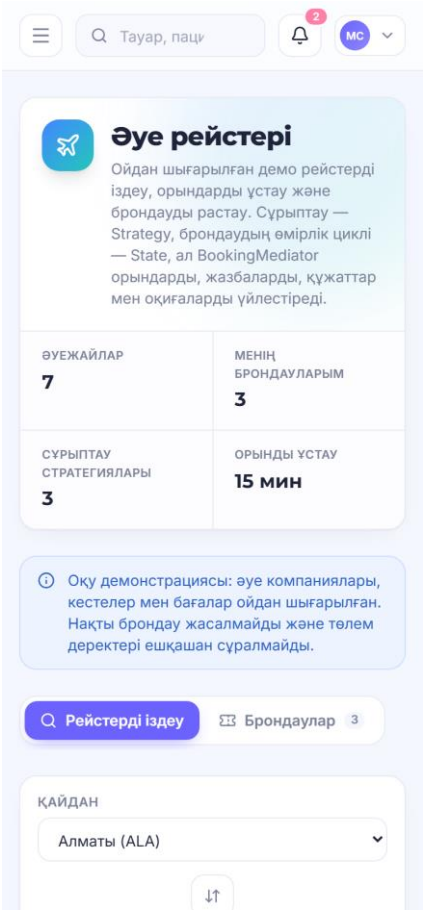
20-кесте — Тапсырмада көрсетілген 12 бизнес-сценарий

№	Сценарий	Тест
1	Тауар жасау	InventoryScenarioTest.createProductValidatesAndRejectsDuplicates
2	Тауарды сыни деңгейге дейін шығару	InventoryScenarioTest.writeOffBelowMinimumTriggersObserverChain
3	Төмен қор хабарландыруы	сол тест (хабарландыру саны +1)
4	Бос уақытқа жазылу	ClinicScenarioTest.bookFreeSlotAndRejectOverlap
5	Қабаттасқан жазылуды қабылдамау	сол тест + concurrentRequestsForTheSameSlotYieldExactlyOneBooking

№	Сценарий	Тест
6	Рейсті бағасы бойынша сұрыптау	FlightScenarioTest.searchSortedByPriceAndByDuration
7	Брондауды растау және болдырмау	FlightScenarioTest.confirmIssuesDocumentAndCancelReleasesSeats
8	Тыйым салынған күй ауысуы	FlightScenarioTest.cancelledBookingCannotBeConfirmed
9	Құжат жасау және бекіту	WorkflowScenarioTest.highValuePurchaseNeedsManagerThenAdmin
10	Себеппен қабылдамау	WorkflowScenarioTest.rejectionRequiresReasonAndNotifiesAuthor
11	Оқиға → хабарландыру	WorkflowScenarioTest / InventoryScenarioTest
12	Басқа рөлдің функциясына тыйым	SecurityIntegrationTest.userRoleCannotAccessInventoryOrAudit

Frontend: Vitest + Testing Library — 23 тест (форматтау және қазақша сөздік, API клиенті және CSRF, тауар формасының валидациясы, компоненттер, кіру беті: қазақша мәтіндер, өрістерді тексеру, демо жол тек өрістерді толтырады, қате құпиясөз хабары). TypeScript тексерісі және production жинағы — қатесіз.

Браузерлік тексеру: бес демо-аккаунттың әрқайсысымен кіру, `/api/auth/me` арқылы рөлді тексеру, шығу және қайта кіру; барлық негізгі беттер 390, 768 және 1280 px ендерінде скриншотпен тексерілді, көлденең прокрутка жоқ.



9-сурет — Мобильді көрініс (390 px): әуе рейстері беті

9. Іске қосу және тоқтату

9.1. Старт.bat

1. Жоба бумасын анықтап, Docker бар-жоғын тексереді; Docker Desktop жабық болса, оны іске қосып, 4 минутқа дейін күтеді.
2. `.env` жоқ болса, `.env.example` негізінде кездейсоқ құпиясөздермен жасайды (бар `.env` ешқашан қайта жазылмайды, құпиясөздер экранға шығарылмайды).
3. `docker compose config` арқылы конфигурацияны тексереді, `docker compose up -d --build` орындайды.
4. Сайт жауап бергенше күтіп, http://localhost:8088 мекенжайын браузерде ашады; қате болса, себебі мен логтарды көру командасын көрсетеді.

9.2. Стоп.bat

Тек осы жобаның контейнерлерін `docker compose stop` арқылы тоқтатады. Дерекқор мен Docker томдары **сақталады** (`down -v` қолданылмайды).

21-кесте — Мекенжайлар және демо-аккаунттар

Не	Мәні
Сайт	http://localhost:8088
Swagger UI	http://localhost:8088/swagger-ui.html
Әкімші	admin / Admin#2026
Менеджер	manager / Manager#2026
Дәрігер	doctor / Doctor#2026
Қызметкер	employee / Employee#2026
Пайдаланушы	user / User#2026

10. Қорғауға арналған демо-сценарий

Ұсынылатын уақыт — 12–15 минут. Екі браузер терезесін ашыңыз (қарапайым және жасырын): біреуінде **employee**, екіншісінде **manager** — хабарландырулардың нақты уақытта келетінін көрсету үшін.

10.1. Негізгі сценарий: Қойма → Observer → Құжат айналымы

22-кесте — Қадамдар және айтылатын паттерндер

№	Әрекет	Не айту керек
1	employee және manager болып кіру	Рөлдер серверде тексеріледі
2	Қойма → тауар → «Қор қозғалысы» → Шығару (ең аз қордан төмен)	Command: `AdjustInventoryCommand`, `CommandInvoker`
3	Тост: «Төмен қор оқиғасы жарияланды»	Strategy (`MinimumLevelStrategy`), Observer (`LowStockEvent`)
4	manager терезесінде тост пен қоңырау	Observer → Strategy (арна) → Factory Method → Abstract Factory → SSE
5	Басқару панеліндегі «Қор аз» көрсеткіші өзгерді	`DashboardRefreshListener`
6	Төмен қор ескертулері → «Өтінім жасап, жіберу»	Facade: `DocumentWorkflowFacade.replenishmentFromAlert`
7	Құжатты ашу: маршрут Жасалды → Тексеру → Менеджер тексеруі → Аяқталды	Chain of Responsibility, State
8	Құжат жасау: 2 400 000 ₸ сатып алу өтінімі — тізбек «Соңғы бекіту» қосады	Тізбектің кеңейтілуі
9	manager: «Менің шешімімді күтуде» → Бекіту	Command, Mediator (үйлестіру журналы)
10	employee «Бекітілді» хабарландыруын алады	Observer
11	Әрекеттер журналы	Command + аудит, `EVENT_*` жазбалары

10.2. Әуе билетін брондау

1. Рейстер → ALA → NQZ іздеу; «Ең арзан / Ең жылдам / Ең ерте» ауыстыру (Strategy).
2. «Таңдау» → орын картасынан орын таңдау → «Орындарды ұстау» → күйі «Күтуде», таймер 15 минут.
3. «Mediator үйлестіру журналы» — әріптестердің тізбегі (BookingFacade → Mediator).
4. «Растау (демо, төлемсіз)» → «Расталған», растау құжаты пайда болады.
5. Зертхана → State: болдырылмаған брондауды растау әрекеті қабылданбайды (HTTP 409).

10.3. Емханаға жазылу

1. user болып кіру → Емхана → «Қабылдауға жазылу»: пациент автоматты таңдалған.
2. Дәрігер мен күнді таңдау → бос емес уақыттар сызылған → жазылу.
3. employee болып сол уақытқа басқа пациентті жазу → «дәрігердің бұл уақытта басқа қабылдауы бар» (Facade + құлыптау).

11. Оқытушының ықтимал сұрақтары және жауаптар

С: Spring singleton мен классикалық Singleton-ның айырмашылығы неде?
Ж: Spring singleton — бір ApplicationContext ішінде бір экземпляр; классикалық Singleton — бір ClassLoader-де бір экземпляр (жеке конструктор, статикалық өріс). Spring нұсқасы тестілеуге ыңғайлы және тәуелділіктерді жасырмайды.

С: Singleton-да неге өзгермелі күй жоқ?
Ж: Себебі бір объектіні барлық ағындар бөліседі. Пайдаланушы немесе транзакция күйі сақталса, деректер араласып кетер еді. Ол үшін әр сұранысқа жаңа `BookingContext` / `WorkflowContext` жасалады.

С: Factory Method пен Abstract Factory неге бір класс емес?
Ж: Олар әртүрлі мәселе шешеді: Factory Method — қандай өңдеушіні жасау (мұрагерлік арқылы бір өнім), Abstract Factory — арнаның үйлесімді компоненттер отбасы. Өңдеуші ішінде фабрика отбасын қолданады.

С: Observer оқиғасы жоғалуы мүмкін бе?
Ж: Иә, процесс commit пен өңдеуші арасында тоқтаса. AFTER_COMMIT кері қайтарылған операция туралы хабарламауға кепілдік береді, бірақ кепілдендірілген жеткізу үшін transactional outbox керек — бұл шектеу құжатталған.

С: Клиент күйді өзі өзгерте ала ма?
Ж: Жоқ. API тек әрекеттерді қабылдайды (`/confirm`, `/cancel`, `/approve`), ал рұқсат етілгенін ағымдағы State объектісі шешеді. Тыйым салынған әрекет 409 қайтарады.

С: Жаңа тексеру ережесін қалай қосасыз?
Ж: Жаңа `DocumentCheckHandler` ішкі класын `@Component` және `@Order` арқылы жазамын — `DocumentProcessingChain` оны автоматты тізбекке қосады, басқа код өзгермейді.

С: Facade неге «құдіретті объектіге» айналмайды?
Ж: Әр ішкі жүйеге бөлек шағын фасад бар (брондау, емхана, құжаттар), ал қадамдардың үйлестірілуі Mediator-ға берілген.

С: Қос брондаудан қалай қорғайсыз?
Ж: Транзакция ішінде дәрігер немесе рейс жолын `SELECT … FOR UPDATE` арқылы құлыптаймын, содан кейін қабаттасуды тексеремін; қосымша — бірегейлік шектеулері және `@Version`. Бұл 4 параллель сұраныспен тесттелген.

С: Неге Command-та undo жоқ?
Ж: Бизнес-ережелер бойынша бекітуді немесе жіберілген хабарландыруды қауіпсіз кері қайтару мүмкін емес; жалған undo пайдаланушыны адастырар еді.

С: Авторизация қатесінің себебі не болды?

Ж: Сервер дұрыс жұмыс істеді; фронтендте кіргеннен кейін `queryClient.clear()` провайдер жазылған `me` сұранысын өшіріп тастады. Енді тек басқа деректер тазаланады.

12. Қорытынды

Жұмыс нәтижесінде бес бизнес-модульді біріктіретін, толық қазақ тіліндегі, жұмыс істейтін full-stack платформа жасалды. Он GoF паттерні жасанды мысалдарда емес, нақты бизнес-сценарийлерде қолданылды және әрқайсысы нақты мәселені шешеді: Singleton ережелерді бір жерде сақтайды; Factory Method пен Abstract Factory хабарландыру арналарын кеңейтуге мүмкіндік береді; Observer модульдерді бір-біріне тәуелсіз етеді; State брондау мен құжаттың өмірлік циклін қорғайды; Strategy алгоритмдерді жұмыс уақытында ауыстырады; Facade күрделі ішкі жүйелерді қарапайым етеді; Mediator күрделі сценарийлерді үйлестіреді; Chain of Responsibility тексерулерді кеңейтуге ыңғайлы етеді; Command әр әрекетті транзакциямен және аудитпен орындайды.

Жоба 63 backend және 23 frontend автоматты тестімен тексерілген, Docker арқылы бір батырмамен іске қосылады, ал «Паттерндер зертханасы» әр паттерннің жұмысын сервердегі нақты кластар арқылы көрсетеді.

Белгілі шектеулер

- SMTP бапталмаса, e-mail жіберілмейді (журналда шынайы көрсетіледі).
- Оқиғалар жадта өңделеді (transactional outbox жоқ).
- Кіру әрекеттерін шектеу және SSE қосылымдары бір сервер жадында сақталады.